# LIBXS

LIBXS is a portable C library providing building blocks for memory operations, numerics, synchronization, and more — with a focus on performance and minimal dependencies. Targets x86-64, AArch64, and RISC-V; requires only a C89 compiler. Originally developed as part of LIBXSMM.

## Functionality

| Domain | Header | Description |
|---|---|---|
| Registry | `libxs_reg.h` | Thread-safe key-value store |
| Predict | `libxs_predict.h` | Fingerprint-guided parameter prediction |
| Malloc | `libxs_malloc.h` | Pool allocator (no system calls at steady) |
| Memory | `libxs_mem.h` | Byte compare, matrix copy/transpose |
| String | `libxs_str.h` | Edit distance, substring, similarity |
| N-gram | `libxs_ngram.h` | Variable-order n-gram counts and backoff |
| Token | `libxs_token.h` | Reversible metatokens and lexical IDs |
| Timer | `libxs_timer.h` | High-resolution timing via calibrated TSC |
| CPUID | `libxs_cpuid.h` | CPU feature detection (SSE..AVX-512, etc.) |
| Utils | `libxs_utils.h` | ISA gates, bit-scan, SIMD helpers |
| Sync | `libxs_sync.h` | Portable atomics, locks, TLS, file locks |
| GEMM | `libxs_gemm.h` | Batched dense GEMM (strided, grouped) |
| Math | `libxs_math.h` | Matrix compare, GCD/LCM, BF16 conversion |
| Hash | `libxs_hash.h` | CRC32, Adler-32, string hashing |
| Perm | `libxs_perm.h` | Shuffle, kd-tree, Hilbert/Morton curves |
| Hist | `libxs_hist.h` | Thread-safe histogram, running statistics |
| Mix | `libxs_mix.h` | Fixed-share mixture over expert banks |
| MHD | `libxs_mhd.h` | Read/write MetaImage (MHD/MHA) files |
| RNG | `libxs_rng.h` | Pseudo-random generation (SplitMix64) |

See also: Fortran Interface, Scripts.

## Build

```
make
```

The library is compiled for SSE4.2 by default but dynamically dispatches to the best ISA available at runtime (up to AVX-512). Use `SSE=0` to compile natively for the build host.

| Variable | Default | Description |
|---|---|---|
| GNU | 1 | GNU GCC-compatible compiler (0: Intel) |
| DBG | 0 | Debug build |
| SYM | 0 | Include debug symbols (-g) |
| SSE | 1 | x86 baseline: 0=native, 1=SSE4.2 (portable) |

CMake is also supported:

```
cmake -S . -B build
cmake --build build
```

To also install the header-only implementation sources, configure with `-DLIBXS_INSTALL_HEADER_ONLY=ON`.

## Installation

Install into a chosen prefix:

```
make -j $(nproc) install PREFIX=$HOME/libxs
```

This installs headers, the Fortran module, the static and shared libraries, and the header-only source tree under `PREFIX`.

Out-of-tree builds are also supported:

```
mkdir /tmp/libxs-build && cd /tmp/libxs-build
make -j $(nproc) -f /path/to/libxs/Makefile
```

## Usage

**Library** — link against `libxs.a` (or `.so`) and include the desired headers:

```
#include <libxs/libxs_mem.h>
#include <libxs/libxs_timer.h>
```

**Header-only** (explicit) — include `libxs_source.h` (no separate library needed). Safe to include from multiple translation units:

```
#include <libxs/libxs_source.h>
```

**Header-only** (implicit) — compile with `-DLIBXS_SOURCE` and any LIBXS public header automatically includes the implementation. No special include order is required. When used through LIBXSTREAM without a pre-built library (`-DLIBXSTREAM_SOURCE`), `LIBXS_SOURCE` is implied automatically.

**Fortran** — use the provided module (documentation):

```
USE :: libxs, ONLY: libxs_memcmp
```

## Samples

| Sample | Description |
|---|---|
| tokenizer | Reversible configurable metatokenizer |
| converse | Extractive summarization via tokenized fprints |
| registry | Registry dispatch microbenchmark |
| predict | Fingerprint-guided parameter prediction |
| rosetta | Hierarchical type discovery on opaque data |
| setdiff | Deterministic set-difference tolerance |
| shuffle | Shuffling strategies comparison |
| scratch | Pool allocator vs system malloc |
| spatial | Stratification, pair counting, kd-tree |
| fprint | Foeppl fingerprint structure/geometry tests |
| memory | Byte comparison, matrix copy, transpose |
| ozaki | Ozaki-scheme low-precision GEMM |
| gemm | Batched DGEMM (strided, pointer, grouped) |
| syrk | Symmetric rank-k/2k update with validation |
| sync | Lock implementation microbenchmarks |

## License

BSD 3-Clause

# LIBXS Domains

## CPU Identification

Header: `libxs_cpuid.h`

Portable CPU feature detection for x86-64, AArch64, and RISC-V targets. Returns an ISA level that can be compared numerically — higher values indicate more capable instruction sets. x86 levels use thermometer ordering: higher numeric value implies all features of lower levels. AVX10/256 sits below AVX512 because it lacks 512-bit vectors.

### ISA Constants

| Constant | Value | Architecture |
|---|---|---|
| `LIBXS_TARGET_ARCH_UNKNOWN` | 0 | Unknown / unsupported |
| `LIBXS_TARGET_ARCH_GENERIC` | 1 | Portable scalar baseline |
| `LIBXS_X86_GENERIC` | 1002 | x86-64 baseline |
| `LIBXS_X86_SSE3` | 1003 | SSE3 |
| `LIBXS_X86_SSE42` | 1004 | SSE4.2 |
| `LIBXS_X86_AVX` | 1005 | AVX |
| `LIBXS_X86_AVX2` | 1006 | AVX2 + FMA |
| `LIBXS_X86_AVX10_256` | 1030 | AVX10.1/256: all features, 256-bit max (Sierra Forest) |
| `LIBXS_X86_AVX512` | 1100 | AVX-512 (F + CD + DQ + BW + VL + VNNI) |
| `LIBXS_X86_AVX512_AMX` | 1105 | AVX-512 + AMX (TILE + INT8 + BF16); Sapphire/Granite Rapids |
| `LIBXS_X86_AVX512_INT8` | 1110 | AVX-512 + AVX-VNNI-INT8 (VPDPBUUD/BSSD) |
| `LIBXS_X86_AVX10_512` | 1200 | AVX10.1/512 + AMX + INT8: full feature set |
| `LIBXS_AARCH64` | 2001 | ARMv8.1 baseline |
| `LIBXS_AARCH64_SVE128` | 2201 | SVE 128-bit |
| `LIBXS_AARCH64_SVE256` | 2301 | SVE 256-bit |
| `LIBXS_AARCH64_SVE512` | 2401 | SVE 512-bit |
| `LIBXS_RV64_MVL128` | 3001 | RISC-V RVV 128-bit |
| `LIBXS_RV64_MVL256` | 3002 | RISC-V RVV 256-bit |
| `LIBXS_RV64_MVL128_LMUL` | 3003 | RISC-V RVV 128-bit (non-unit LMUL) |
| `LIBXS_RV64_MVL256_LMUL` | 3004 | RISC-V RVV 256-bit (non-unit LMUL) |
| `LIBXS_X86_ALLFEAT` | 1999 | x86 sentinel (all features) |
| `LIBXS_AARCH64_ALLFEAT` | 2999 | AArch64 sentinel (all features) |
| `LIBXS_RV64_ALLFEAT` | 3999 | RISC-V sentinel (all features) |

### Types

```
typedef struct libxs_cpuid_t {
  char model[256];   /* CPU model name from OS */
  int constant_tsc;  /* non-zero if TSC is invariant */
} libxs_cpuid_t;
```

### Functions

```
int libxs_cpuid(libxs_cpuid_t* info);
```

Detect the ISA level of the current platform. Optionally fills `info` with the CPU model name and TSC capability. Returns a constant from the table above.

```
const char* libxs_cpuid_name(int id);
```

Returns a human-readable string for the given ISA constant, e.g., `"avx2"`.

```
int libxs_cpuid_id(const char* name);
```

Translates a name (e.g., `"avx512"`) to the corresponding ISA constant.

```
int libxs_cpuid_vlen(int id);
```

Returns the SIMD vector length in bytes for the given ISA level (0 for scalar-only targets).

```
int libxs_cpuid_amx_enable(void);
```

Request AMX tile state (XTILE_DATA) from the OS. Must be called before any AMX tile instruction. Returns `EXIT_SUCCESS` if tiles are ready (or were already enabled), `EXIT_FAILURE` on unsupported platform or OS refusal. Not called automatically by `libxs_cpuid` to avoid the per-thread XSAVE overhead for non-AMX workloads.

## GEMM: Matrix Multiplication

Header: `libxs_gemm.h` Fortran: `USE LIBXS` (libxs/libxs.f)

Batched general matrix-matrix multiplication (GEMM) and symmetric rank-k/2k updates. Operations are expressed as C := alpha * op(A) * op(B) + beta * C, where op() is an optional transpose. Kernels are dispatched via MKL JIT, LIBXSMM, or BLAS; a built-in default kernel (auto-vectorized) is used as fallback.

### Types

```
typedef struct libxs_gemm_shape_t {
  libxs_data_t datatype;
  char transa, transb;
  int m, n, k, lda, ldb, ldc;
  double alpha, beta;
} libxs_gemm_shape_t;
```

GEMM shape: problem geometry, transpose flags, and scalar coefficients. Serves as registry key when caching dispatched configurations. The key is hashed byte-wise, hence dispatch builds a zero-initialized copy rather than hashing a caller-supplied struct (the two chars leave padding before `m`).

```
typedef struct libxs_gemm_config_t {
  libxs_gemm_dblas_t dgemm_blas;
  libxs_gemm_sblas_t sgemm_blas;
  libxs_gemm_djit_t dgemm_jit;
  libxs_gemm_sjit_t sgemm_jit;
  libxs_gemm_xfn_t xgemm;
  void* jitter;
  libxs_gemm_flags_t flags;
  int warmup;
  libxs_gemm_shape_t shape;
} libxs_gemm_config_t;
```

Configuration holding dispatched GEMM kernels. Kernel priority:

1. JIT kernel (dgemm_jit/sgemm_jit + jitter),
2. XGEMM kernel (xgemm),
3. BLAS kernel (dgemm_blas/sgemm_blas) – always non-NULL after dispatch (falls back to built-in auto-vectorized C code).

warmup counts dispatches of this shape and is maintained by dispatch (see LIBXS_GEMM_JIT_WARMUP).

```
typedef enum libxs_gemm_flags_t {
  LIBXS_GEMM_FLAGS_DEFAULT = 0,
  LIBXS_GEMM_FLAG_NOLOCK = 1,
  LIBXS_GEMM_FLAG_OWNJIT = 2
} libxs_gemm_flags_t;
```

Flags controlling batch synchronization. Set LIBXS_GEMM_FLAG_NOLOCK when no duplicate C pointers exist across the batch. LIBXS_GEMM_FLAG_OWNJIT is set by dispatch and marks the single config owning `jitter`; aliasing configs (double-dispatch, and every caller copy) leave it clear so the handle is released exactly once.

### Dispatch

```
libxs_gemm_config_t* libxs_gemm_dispatch(
  libxs_data_t datatype, char transa, char transb,
  int m, int n, int k, int lda, int ldb, int ldc,
  const void* alpha, const void* beta,
  void* registry /* = NULL */);
```

Inline function that selects the backend at compile time:

- MKL JIT (if mkl.h is included before libxs_gemm.h),
- LIBXSMM (if libxsmm.h is included),
- BLAS dgemm/sgemm (if ___BLAS, ___MKL, or MKL_H is defined),
- built-in default otherwise.

On registry hit, returns a pointer to the cached config (hash probe only). On miss, dispatches a new kernel and stores it. If registry is NULL, an internal registry is used.

The detection itself is exposed, so a caller assembling shapes by hand can reach `libxs_gemm_dispatch_rt` without repeating it:

```
void libxs_gemm_backend_init(libxs_gemm_backend_t* backend);
```

It zero-initializes backend and fills the pointers available at the caller's compile time. Unavailable backends are left NULL, i.e., dispatch falls through to the built-in default kernel.

**Config ownership (registry pointer or caller copy)** Every dispatch function comes in two flavors. The unsuffixed C form returns a pointer to the registry-owned config; the `_cpy` form populates a caller-owned config instead:

```
int libxs_gemm_dispatch_cpy(
  libxs_gemm_config_t* config,
  libxs_data_t datatype, char transa, char transb,
  int m, int n, int k, int lda, int ldb, int ldc,
  const void* alpha, const void* beta,
  void* registry /* = NULL */);

int libxs_gemm_config_cpy(
  libxs_gemm_config_t* config, const libxs_gemm_config_t* cached);
```

Both return nonzero if config was populated (config is left untouched otherwise, hence zero-initialize it before the call). `libxs_gemm_config_cpy` converts a pointer obtained from any dispatch function into a caller-owned config.

Pick the flavor by lifetime, not by convenience:

| Flavor | Use when |
|---|---|
| pointer | the config is cached across many calls and should pick up a kernel once JIT warm-up completes |
| copy | flags are modified, per-thread state is wanted, or dispatch happens per call anyway |

A copy is a snapshot. It is not upgraded in place when warm-up completes later, so re-dispatch to pick up a kernel. Writing `flags` through a registry pointer affects every user of that shape, hence a copy is the safe place for `LIBXS_GEMM_FLAG_NOLOCK`. `LIBXS_GEMM_FLAG_OWNJIT` is cleared in a copy: the JIT handle stays owned by the registry, and `libxs_gemm_release` is a no-op for the copy (release the registry, or the config the registry returned).

The Fortran spelling is mirrored, with the historical difference that `libxs_gemm_dispatch` is already the copy flavor there:

| Operation | C pointer | C copy | Fortran pointer | Fortran copy |
|---|---|---|---|---|
| GEMM | `libxs_gemm_dispatch` | `libxs_gemm_dispatch_cpy` | `libxs_gemm_dispatch_ptr` | `libxs_gemm_dispatch` |
| SYR2K | `libxs_syr2k_dispatch` | `libxs_syr2k_dispatch_cpy` | `libxs_syr2k_dispatch` | `libxs_syr2k_dispatch_cpy` |
| SYRK | `libxs_syrk_dispatch` | `libxs_syrk_dispatch_cpy` | `libxs_syrk_dispatch` | `libxs_syrk_dispatch_cpy` |

All of the above are inlines (or Fortran wrappers) that detect the backend at the caller's compile time. Backends are never a runtime argument there, which is what keeps LIBXS decoupled from MKL, LIBXSMM, and BLAS. To supply backend pointers explicitly, use the `_rt` forms, which are the only dispatch symbols exported by the library itself:

| Operation | Explicit backend (library symbol) |
|---|---|
| GEMM | `libxs_gemm_dispatch_rt` (takes shape structs) |
| SYR2K | `libxs_syr2k_dispatch_rt` |
| SYRK | `libxs_syrk_dispatch_rt` |

```
typedef struct libxs_gemm_backend_t {
  libxs_jit_create_dgemm_t jit_create_dgemm;
  libxs_jit_get_dgemm_t   jit_get_dgemm;
  libxs_jit_create_sgemm_t jit_create_sgemm;
  libxs_jit_get_sgemm_t   jit_get_sgemm;
  libxs_xgemm_dispatch_t  xgemm_dispatch;
  libxs_gemm_dblas_t dgemm_blas;
  libxs_gemm_sblas_t sgemm_blas;
} libxs_gemm_backend_t;

libxs_gemm_config_t* libxs_gemm_dispatch_rt(
  const libxs_gemm_shape_t* shape,
  const libxs_gemm_shape_t* kernel_shape,
  const libxs_gemm_backend_t* backend,
  void* registry);
```

Non-inline function that accepts backend and shape structs. shape: full problem shape (registry key, stored in config). kernel_shape: actual kernel dimensions (may differ, e.g., tight ldc for scratch). NULL means same as shape. If kernel_shape differs from shape, the kernel is looked up under kernel_shape first (double-dispatch), avoiding redundant code generation. backend: function pointers for backends. NULL means built-in default only. Same registry semantics as above. `LIBXS_GEMM_BACKEND` can restrict the starting point of the runtime fallback chain: 0 = automatic/default, 1 = MKL JIT, 2 = LIBXSMM, 3 = BLAS/MKL, 4 = built-in fallback. Choices 1-3 still fall through to lower-priority backends if the requested backend is not supplied or cannot dispatch the shape.

Backend callback signatures (MKL-compatible):

```
jit_create_dgemm: int(void** jitter, int layout, int transa,
                      int transb, int m, int n, int k,
                      double alpha, int lda, int ldb,
                      double beta, int ldc)
                  Return: MKL_JIT_SUCCESS (0) or
                          MKL_NO_JIT (1) on success,
                          MKL_JIT_ERROR (2) on failure.

jit_get_dgemm:    void*(void* jitter)
                  Return kernel function pointer.

xgemm_dispatch:   libxs_gemm_xfn_t(
                      libxs_xgemm_shape_t shape /* by value */,
                      unsigned int gemm_flags,
                      unsigned int prefetch_flags)
                  Layout-compatible with libxsmm_dispatch_gemm.
                  shape: m,n,k,lda,ldb,ldc then a_in_type,
                         b_in_type, out_type, comp_type.
                  gemm_flags: bit 0 = transa, bit 1 = transb,
                         bit 2 = beta==0.
                  Dispatch is attempted only for alpha==1 with
                  beta in {0,1}; other scalars skip this backend.
```

```
USE LIBXS_JIT
rc = libxs_gemm_dispatch(config, LIBXS_DATATYPE_F64,
     &  'N', 'N', m, n, k, lda, ldb, ldc, alpha, beta)
```

The LIBXS_JIT adapter module provides the same dispatch API but fills in BLAS (and MKL JIT when compiled with __MKL) automatically. Requires BLAS at link time.

```
USE LIBXS
rc = libxs_gemm_dispatch(config, datatype, transa, transb,
     &  m, n, k, lda, ldb, ldc, alpha, beta,
     &  jit_create_dgemm=..., jit_get_dgemm=...,
     &  dgemm_blas=..., registry=...)
```

All backend arguments are OPTIONAL C_FUNPTR (named arguments). Returns nonzero on success (dispatch produced a callable config). The config is populated from the registry-owned copy.

### Single-Kernel Call

```
void libxs_gemm_call(
  const libxs_gemm_config_t* config,
  const void* a, const void* b, void* c);
```

Call the dispatched GEMM kernel (JIT > XGEMM > BLAS fallback). The caller must ensure config is non-NULL (dispatch succeeded).

### Release

```
void libxs_gemm_release(libxs_gemm_config_t* config);
```

Release resources (e.g., MKL jitter handle) held by config, and clear the JIT kernel such that config falls back to XGEMM/BLAS. A handle can be shared by several configs (double dispatch), hence only the owning config (LIBXS_GEMM_FLAG_OWNJIT, set by dispatch) releases it and a handle is released exactly once. A dispatched config is owned by the registry, i.e., releasing it affects every user of that registry: release at teardown, or rely on libxs_gemm_release_registry.

```
void libxs_gemm_release_registry(libxs_registry_t* registry);
```

Release all configs in a registry, then destroy the registry.

### Pointer-Array Batch

```
void libxs_gemm_batch(
  const void* a_array[], const void* b_array[], void* c_array[],
  int batchsize, const libxs_gemm_config_t* config);

void libxs_gemm_batch_task(
  const void* a_array[], const void* b_array[], void* c_array[],
  int batchsize, const libxs_gemm_config_t* config,
  int tid, int ntasks);
```

Batch of GEMMs from pointer arrays. The _task variant splits work across ntasks threads (tid = 0..ntasks-1).

#### Index/Strided Batch

```
void libxs_gemm_index(
  const void* a, const int stride_a[],
  const void* b, const int stride_b[],
        void* c, const int stride_c[],
  int index_stride, int index_base,
  int batchsize, const libxs_gemm_config_t* config);

void libxs_gemm_index_task(
  const void* a, const int stride_a[],
  const void* b, const int stride_b[],
        void* c, const int stride_c[],
  int index_stride, int index_base,
  int batchsize, const libxs_gemm_config_t* config,
  int tid, int ntasks);
```

Batch of GEMMs from element-offset index arrays into contiguous buffers. index_base: 0 (C) or 1 (Fortran). index_stride: byte stride between consecutive index entries (sizeof(int) for packed arrays, 0 for constant-stride mode).

### SYR2K / SYRK

Symmetric rank-2k and rank-k updates built on top of GEMM dispatch.

For small problems (n <= LIBXS_GEMM_BLOCK_M and k <= LIBXS_GEMM_BLOCK_K), the dispatched kernel handles the full GEMM in one call. For larger problems, the implementation tiles the output into blocks and accumulates along K. Full-size tiles use the dispatched JIT kernel; remainder tiles fall back to BLAS (if available) or the built-in default. Scratch memory is thread-local.

```
libxs_gemm_config_t* libxs_syr2k_dispatch(
  libxs_data_t datatype, int n, int k, int lda, int ldb, int ldc,
  void* registry /* = NULL */);

libxs_gemm_config_t* libxs_syrk_dispatch(
  libxs_data_t datatype, int n, int k, int lda, int ldc,
  void* registry /* = NULL */);

int libxs_syr2k_dispatch_cpy(
  libxs_gemm_config_t* config,
  libxs_data_t datatype, int n, int k, int lda, int ldb, int ldc,
  void* registry /* = NULL */);

int libxs_syrk_dispatch_cpy(
  libxs_gemm_config_t* config,
  libxs_data_t datatype, int n, int k, int lda, int ldc,
  void* registry /* = NULL */);
```

Dispatch a GEMM config for SYR2K/SYRK. The problem shape (n, k, lda, ldb, ldc) is stored in the config and used by the call functions, while the dispatched kernel is the SYRK tile, i.e., the blocking (LIBXS_GEMM_BM/BN/BK) is applied inside. These are inlines and detect the backend at the caller's compile time, exactly like libxs_gemm_dispatch. The `_cpy` flavors populate a caller-owned config (see Config ownership).

To pass backend pointers explicitly, use the runtime forms:

```
libxs_gemm_config_t* libxs_syr2k_dispatch_rt(
  libxs_data_t datatype, int n, int k, int lda, int ldb, int ldc,
  const libxs_gemm_backend_t* backend /* = NULL */,
  void* registry /* = NULL */);

libxs_gemm_config_t* libxs_syrk_dispatch_rt(
  libxs_data_t datatype, int n, int k, int lda, int ldc,
  const libxs_gemm_backend_t* backend /* = NULL */,
  void* registry /* = NULL */);
```

Fortran mirrors both: LIBXS_JIT auto-detects, module LIBXS accepts OPTIONAL backend function pointers.

```
ptr = libxs_syrk_dispatch(LIBXS_DATATYPE_F64, n, k, lda, ldc,
     &  jit_create_dgemm=C_FUNLOC(mkl_cblas_jit_create_dgemm),
     &  jit_get_dgemm=C_FUNLOC(mkl_jit_get_dgemm_ptr),
     &  dgemm_blas=C_FUNLOC(DGEMM))

rc = libxs_syrk_dispatch_cpy(config, LIBXS_DATATYPE_F64,
     &  n, k, lda, ldc)
```

```
void libxs_syr2k(
  const libxs_gemm_config_t* config, char uplo,
  double alpha, double beta,
  const void* a, const void* b, void* c);

void libxs_syr2k_task(
  const libxs_gemm_config_t* config, char uplo,
  double alpha, double beta,
  const void* a, const void* b, void* c,
  int tid, int ntasks);
```

C := alpha*(A$B\hat{}T + B$A^T) + beta*C. Only the triangle specified by uplo ('U' or 'L') is written. All dimensions and leading dimensions come from the dispatched config.

```
void libxs_syrk(
  const libxs_gemm_config_t* config, char uplo,
  double alpha, double beta,
  const void* a, void* c);

void libxs_syrk_task(
  const libxs_gemm_config_t* config, char uplo,
  double alpha, double beta,
  const void* a, void* c,
  int tid, int ntasks);
```

C := alpha$A$A^T + beta*C. Only the triangle specified by uplo ('U' or 'L') is written.

The _task variants partition work across ntasks threads. Each thread operates on independent output blocks; no locking is required. Thread-local scratch buffers are used internally.

### Compile-Time Tuning

The block sizes used for tiled SYRK/SYR2K can be overridden at compile time via preprocessor defines:

```
LIBXS_GEMM_BM        Row block size    (default: 24)
LIBXS_GEMM_BN        Column block size (default: 48)
LIBXS_GEMM_BK        K-direction block (default: 128)
```

Problems fitting within these limits use a single specialized kernel call (MKL JIT or LIBXSMM when available). The same values are the SYRK tile dispatched as the kernel shape, hence they decide which kernel the arithmetic-intensity gate (LIBXS_GEMM_JIT_MAX) sees.

```
LIBXS_GEMM_NWARMUP   JIT warm-up counters (default: 4096)
```

Number of slots in the table holding the per-shape reuse counters (must be a power of two). Shapes share slots by hash; a collision is detected by an identity tag stored alongside the counter, and the losing shape simply restarts counting rather than inheriting foreign progress. Sizing this below the number of distinct shapes in flight therefore degrades to later JIT, never to a wrong kernel.

### Environment Variables

```
LIBXS_GEMM_BM=N         Row block size for tiled SYRK/SYR2K
                        (default: 24).
LIBXS_GEMM_BN=N         Column block size (default: 48).
LIBXS_GEMM_BK=N         K-direction block size (default: 128).
LIBXS_GEMM_BACKEND=N    Select runtime backend chain start:
                        0 auto (default), 1 MKL JIT,
                        2 LIBXSMM, 3 BLAS, 4 built-in fallback.
LIBXS_GEMM_JIT_MAX=N    Arithmetic-intensity threshold for JIT
                        dispatch. JIT/LIBXSMM kernels are only
                        generated when the kernel shape's AI
                        (flops/bytes) is below N (default: 7,
                        roughly the AI of an 80x80x80 kernel).
                        Set to 0 to disable JIT entirely.
LIBXS_GEMM_JIT_WARMUP=N Number of calls a shape must
                        accumulate before JIT compilation is
                        attempted. Shapes called fewer than N
                        times use the BLAS fallback; only shapes
                        with proven reuse pay the JIT compile
                        cost. Reuse is counted per dispatch, hence
                        the first shape of an empty registry is
                        compiled immediately (a caller dispatching
                        only once cannot accumulate calls).
                        Counted in a fixed-size table, not in the
                        registry (see LIBXS_GEMM_NWARMUP); once a
                        shape's JIT has been attempted it is never
                        attempted again, whether it succeeded or
                        was refused by LIBXS_GEMM_JIT_MAX.
                        Set to 0 or 1 to compile immediately
                        on first miss (default: 8, clamped to 254).
LIBXS_GEMM_PRINT=N      Print dispatch info every N-th call
                        to stderr (requires compile-time gate).
LIBXS_SYRK_PRINT=N      Print DSYRK/DSYR2K BLAS fallback calls
                        to stderr (requires compile-time gate).
```

### Example (C)

```c
#include <libxs/libxs_gemm.h>

libxs_registry_t* reg = libxs_registry_create();

/* dispatch once per unique shape */
const libxs_gemm_config_t* cfg = libxs_syr2k_dispatch(
  LIBXS_DATATYPE_F64, n, k, lda, ldb, ldc, reg);

/* call many times (registry hit = hash probe only) */
for (batch = 0; batch < nbatches; ++batch) {
  libxs_syr2k(cfg, 'U', 0.5, 0.0, a[batch], b[batch], c[batch]);
}

libxs_gemm_release_registry(reg);
```

### Example (Fortran)

```fortran
USE :: LIBXS

TYPE(libxs_gemm_config_t) :: config
INTEGER(C_INT) :: rc

rc = libxs_gemm_dispatch(config, LIBXS_DATATYPE_F64,
     &  'N', 'N', m, n, k, lda, ldb, ldc, alpha, beta,
     &  dgemm_blas=C_FUNLOC(DGEMM))

IF (0 /= rc) THEN
```

```
  CALL libxs_gemm_call(config, C_LOC(a), C_LOC(b), C_LOC(c))
END IF
! rc /= 0 guarantees libxs_gemm_call will succeed
! (JIT, XGEMM, or BLAS/built-in fallback)
```

## Hashing and Checksums

Header: `libxs_hash.h`

Hash functions for buffers, fixed-width keys, and strings. The default hash uses hardware-accelerated CRC32-C (SSE4.2) with a software fallback.

### Buffer Hashing

```
unsigned int libxs_hash(const void* data, unsigned int size,
  unsigned int seed);
```

Produce a 32-bit hash from a byte buffer of the given size. NULL buffer is accepted (returns seed). Uses CRC32-C when SSE4.2 is available, otherwise a software table-based CRC32.

### Fixed-Width Hashing

```
unsigned int libxs_hash8(unsigned int data);
unsigned int libxs_hash16(unsigned int data);
unsigned int libxs_hash32(unsigned long long data);
```

Hash an 8-, 16-, or 32-bit value. The input is split into a data part and an implicit seed derived from the remaining bits. The result is masked to the corresponding width (8, 16, or 32 bits).

### String Hashing

```
unsigned long long libxs_hash_string(const char string[]);
```

64-bit hash of a character string. Short strings (up to 8 bytes) are stored directly; longer strings use two CRC32 passes combined into a 64-bit result. NULL-string is accepted (returns zero).

### CRC32 (ISO 3309)

```
unsigned int libxs_hash_iso3309(const void* data,
  unsigned int size, unsigned int seed);
```

CRC-32 using the ISO 3309 polynomial (used by PNG, gzip, and similar formats). Software-only, no hardware acceleration. Pre-condition and post-XOR with 0xFFFFFFFF are the caller's responsibility. NULL buffer is accepted.

### Adler-32

```
unsigned int libxs_adler32(const void* data,
  unsigned int size, unsigned int seed);
```

Adler-32 checksum (the variant used by zlib). The standard initial seed is 1. NULL buffer is accepted.

## Histogram

Header: `libxs_hist.h`

Thread-safe histogram with running statistics. Buckets by `vals[0]`; additional values per entry track user-defined statistics.

### Types

```
typedef struct libxs_hist_t libxs_hist_t;  /* opaque */
typedef struct libxs_span_t libxs_span_t;  /* opaque */

typedef void (*libxs_hist_update_t)(double* dst, const double* src, int count);
typedef void (*libxs_hist_fold_t)(void* ctx, double* queue, int nvals, int nsamples);
```

An update function reduces one value into one bucket, pairwise and in an order that must not matter. A fold instead sees a whole batch of samples at once and keeps state of its own, which is what a bucket cannot express — relating samples to one another rather than summarizing them independently.

```
typedef struct libxs_hist_info_t {
  const int* buckets;   /* per-bucket counts [nbuckets] */
  const double* vals;   /* per-bucket values [nbuckets * nvals] */
  const double* sum;    /* running total per value [nvals] */
  double range[2];      /* [min, max] of vals[0] */
  int nbuckets, nvals, nsamples;
} libxs_hist_info_t;
```

`sum` is accumulated on push and is therefore independent of the update functions: it is a true total even where a bucket keeps a mean, a minimum or a maximum. `nsamples` equals the sum of the bucket counts.

### Functions

```
libxs_hist_t* libxs_hist_create(int nbuckets, int nvals,
  const libxs_hist_update_t update[], libxs_hist_fold_t fold, void* ctx);
void libxs_hist_destroy(libxs_hist_t* hist);
```

Create/destroy. `update[nvals]` specifies per-slot accumulation (NULL defaults to avg). `fold` is optional (NULL for none) and receives `ctx` unchanged; the histogram only borrows it, so it must outlive the histogram. Destroy accepts NULL.

```
void libxs_hist_push(libxs_lock_t* lock,
  libxs_hist_t* hist, const double vals[]);
```

Insert one sample. Samples are queued and folded into the buckets in batches, so a histogram is exact while a batch fits and summarizes only once it is full. Re-bins automatically if `vals[0]` exceeds the established range.

```
void libxs_hist_query(libxs_lock_t* lock,
  const libxs_hist_t* hist, libxs_hist_info_t* info);
```

Query statistics, folding whatever is still queued. Repeated calls report the same result rather than folding the same samples twice.

```
void libxs_hist_query_percentile(libxs_lock_t* lock,
  const libxs_hist_t* hist, double vals[], double percentile);
void libxs_hist_query_median(libxs_lock_t* lock,
  const libxs_hist_t* hist, double vals[]);
void libxs_hist_query_mode(libxs_lock_t* lock,
  const libxs_hist_t* hist, double vals[]);
```

Values at a percentile [0..1], at the median (0.5), or of the most populated bucket. All report per-bucket aggregates, blending only towards a populated neighbour, so the result describes samples that were actually observed. The mode is preferable for a multi-modal distribution, where a percentile can fall between clusters; its `vals[0]` is the bucket's upper bound rather than an aggregate.

```
void libxs_hist_print(FILE* ostream, const libxs_hist_t* hist,
  const int prec[], const char fmt[], ...);
```

Print to stream. `prec[k]` controls precision; negative skips output. NULL ostream/fmt accepted.

### Interval Union

```
libxs_span_t* libxs_span_create(int nsegments);
void libxs_span_destroy(libxs_span_t* span);
void libxs_span_push(libxs_span_t* span, double begin, double end);
double libxs_span_total(const libxs_span_t* span, int* inexact);

void libxs_hist_fold_union(void* ctx, double* queue, int nvals, int nsamples);
```

The time a set of intervals covers, as opposed to the sum of their lengths, which counts overlapping intervals more than once. `nsegments` bounds how far out of order intervals may arrive, not how many there may be: an interval beginning before what has already been retired is added whole and counted in `inexact`, which makes the total an upper bound. It never understates, so a ratio of summed length over covered time is a lower bound on how many intervals overlapped.

`libxs_hist_fold_union` is a ready-made `libxs_hist_fold_t` that expects `ctx` to be a `libxs_span_t*` and reads the last two values of a sample as `(begin, end)`; pass it to `libxs_hist_create` to obtain both a distribution of durations and the time they covered from one push.

Values must share an origin a double can hold exactly. Wall-clock nanoseconds do not — far above 2^53 the spacing between representable values is hundreds of nanoseconds, enough to make adjacent intervals appear to touch — so subtract an epoch first.

### Update Functions

```
void libxs_hist_update_avg(double* dst, const double* src, int count);
void libxs_hist_update_add(double* dst, const double* src, int count);
void libxs_hist_update_min(double* dst, const double* src, int count);
void libxs_hist_update_max(double* dst, const double* src, int count);
```

- `avg` — Welford online mean: `*dst += (*src - *dst) / count`
- `add` — sum: `*dst += *src`
- `min` — `*dst = min(*dst, *src)`
- `max` — `*dst = max(*dst, *src)`

Custom callbacks use the same signature; `count` enables online algorithms without external state. A callback that needs the whole sample, or state outside a bucket, is a fold rather than an update.

### Memory Allocation

Header: `libxs_malloc.h`

Pool-based memory allocator designed for steady-state performance: after an initial warm-up phase, allocations are served from a recycled pool without system calls.

#### Types

```
typedef struct libxs_malloc_info_t {
  size_t size;  /* allocated size in bytes */
} libxs_malloc_info_t;

typedef struct libxs_malloc_pool_hist_t {
  size_t count;         /* allocations in this bucket */
  size_t nreuses;       /* chunk reused without reallocation */
  size_t ngrows;        /* chunk reallocated (too small) */
  size_t nevicts_age;   /* evictions triggered by age */
  size_t nevicts_limit; /* evictions triggered by memory limit */
} libxs_malloc_pool_hist_t;

typedef struct libxs_malloc_pool_info_t {
  size_t used;      /* memory currently in use (sum of requested sizes) */
  size_t size;      /* total allocated memory (sum of actual chunk sizes) */
  size_t peak;      /* peak memory consumption */
  size_t nactive;   /* pending (not yet freed) allocations */
  size_t nmallocs;  /* total allocation count */
  libxs_malloc_pool_hist_t hist[6]; /* per-bucket histogram */
} libxs_malloc_pool_info_t;
```

The histogram uses 6 logarithmic buckets keyed on requested size: <1M, 1-4M, 4-16M, 16-64M, 64-256M, >=256M. Each bucket tracks how allocations in that size class were served (reuse vs. grow) and how many were evicted by the pool's memory-pressure heuristics.

```
typedef struct libxs_malloc_pool_t libxs_malloc_pool_t; /* opaque */

typedef void* (*libxs_malloc_fn)(size_t size);
typedef void  (*libxs_free_fn)(void* pointer);

typedef void* (*libxs_malloc_xfn)(size_t size, const void* extra);
typedef void  (*libxs_free_xfn)(void* pointer, const void* extra);
```

#### Pool Management

```
libxs_malloc_pool_t* libxs_malloc_pool(libxs_malloc_fn malloc_fn, libxs_free_fn free_fn);
```

Create a memory pool. If `malloc_fn` and `free_fn` are both NULL, the standard `malloc`/`free` are used. Both must be NULL or both must be non-NULL.

```
libxs_malloc_pool_t* libxs_malloc_xpool(libxs_malloc_xfn malloc_fn, libxs_free_xfn free_fn,
  int max_nthreads);
```

Create a memory pool with extended allocator functions that receive a per-thread extra argument (see `libxs_malloc_arg`). The `max_nthreads` parameter determines the size of the internal per-thread argument table (indexed by `libxs_tid`). Both function pointers must be non-NULL and `max_nthreads` must be positive; otherwise NULL is returned.

```
void libxs_malloc_arg(libxs_malloc_pool_t* pool, const void* extra);
```

Set the per-thread extra argument for an extended pool (created by `libxs_malloc_xpool`). The pointer is stored at the calling thread's slot (`libxs_tid() % max_nthreads`) and passed to the registered `malloc_xfn`/`free_xfn` on subsequent allocations and frees from this thread. No-op for standard pools. Access is lock-free since each thread writes only its own slot.

```
void libxs_free_pool(libxs_malloc_pool_t* pool);
```

Destroy the pool and release all associated memory. Accepts NULL.

```
int libxs_malloc_pool_info(const libxs_malloc_pool_t* pool, libxs_malloc_pool_info_t* info);
```

Query aggregate pool statistics including the per-bucket histogram.

```
void libxs_malloc_pool_print(FILE* ostream, const char prefix[],
  const libxs_malloc_pool_t* pool);
```

Print pool statistics and per-bucket histogram to `ostream`. Each non-empty bucket is printed on its own line showing count, reuse, grow, and eviction numbers. The `prefix` string (may be NULL) is prepended to each line. No output is produced if the pool has zero allocations or `ostream` is NULL.

```
libxs_malloc_pool_t* libxs_default_pool(void);
```

Access the default host memory pool (created at `libxs_init`). Printed automatically at `libxs_finalize` when `LIBXS_VERBOSE` >= 4.

#### General Allocation

```
void* libxs_malloc(libxs_malloc_pool_t* pool, size_t size, int alignment);
```

Allocate `size` bytes from the given `pool`. `LIBXS_MALLOC_AUTO` uses automatic alignment with inline metadata. `LIBXS_MALLOC_NATIVE` preserves the allocator's native pointer (out-of-band metadata via registry). Values greater than 1 are interpreted as explicit alignment in Bytes (inline metadata). Returns NULL on failure or if `pool` is NULL.

```
void libxs_free(void* pointer);
```

Return memory to its originating pool (derived internally). Accepts NULL.

```
int libxs_malloc_info(const void* pointer, libxs_malloc_info_t* info);
```

Query the size of an allocation made with `libxs_malloc`. The pool is derived internally.

#### Fixed-Size Pool

A lightweight fixed-size pool for scenarios where the element size is known at initialization time.

```
void libxs_pmalloc_init(size_t size, size_t* num,
  void* pool[], void* storage);
```

Partition `storage` into `*num` chunks of `size` bytes and register them in `pool`.

```
void* libxs_pmalloc(void* pool[], size_t* num);
void* libxs_pmalloc_lock(void* pool[], size_t* num,
  libxs_lock_t* lock);
```

Pop one chunk from the pool. The `_lock` variant uses a caller-provided lock; the plain variant uses an internal lock.

```
void libxs_pfree(void* pointer, void* pool[], size_t* num);
void libxs_pfree_lock(void* pointer, void* pool[],
  size_t* num, libxs_lock_t* lock);
```

Push a chunk back into the pool.

### Math Utilities

Header: `libxs_math.h`

### Matrix Difference

```
typedef struct libxs_matdiff_t {
  double norm1_abs, norm1_rel;  /* one-norm */
  double normi_abs, normi_rel;  /* infinity-norm */
  double normf_rel;             /* Frobenius-norm (relative) */
  double linf_abs, linf_rel;    /* max difference (abs/rel at same element) */
  double l2_abs, l2_rel, rsq;   /* L2-norm and R-squared */
  double l1_ref, min_ref, max_ref, avg_ref, var_ref;  /* reference stats */
  double l1_tst, min_tst, max_tst, avg_tst, var_tst;  /* test stats */
  double diag_min_ref, diag_max_ref;   /* diagonal min/max (reference) */
  double diag_min_tst, diag_max_tst;   /* diagonal min/max (test) */
  double v_ref, v_tst;          /* values at max-diff location */
  double w;                     /* cumulative weight for online mean */
  int m, n, i, r;               /* location and reduction count */
} libxs_matdiff_t;
```

The fields linf_abs, linf_rel, and v_ref/v_tst always refer to the same element. For complex types, statistics use modulus; differences use complex absolute error.

The `w` field accumulates element count (m*n) across reductions. `libxs_matdiff_reduce` uses weighted online mean (West/Welford) for avg_ref/avg_tst: collapses to plain Welford when all matrices are the same size; gives the exact element-weighted grand mean otherwise.

```
int libxs_matdiff(libxs_matdiff_t* info,
  libxs_data_t datatype, int m, int n,
  const void* ref, const void* tst,
  const int* ldref, const int* ldtst);
```

Compute scalar differences between two matrices. Supports all real and integer `libxs_data_t` types, plus `LIBXS_DATATYPE_C64` and `LIBXS_DATATYPE_C32` (interleaved complex; dimensions refer to complex elements).

```
double libxs_matdiff_epsilon(const libxs_matdiff_t* input);
```

Combined error margin from absolute and relative norms. Optionally logs to file via `LIBXS_MATDIFF` env var.

```
int libxs_matdiff_combine(libxs_matdiff_t* output,
  const libxs_matdiff_t* input);
```

Combine two single-matrix infos (`ref=NULL`) into a meta-diff. Per-side statistics are exact; `linf_abs` is the mean shift, `l2_abs` a statistical bound.

```
void libxs_matdiff_reduce(libxs_matdiff_t* output,
  const libxs_matdiff_t* input);
void libxs_matdiff_clear(libxs_matdiff_t* info);
```

Worst-case reduction of matdiff results. Initialize with `libxs_matdiff_clear`.

```
double libxs_matdiff_posdef(const libxs_matdiff_t* info);  /* inline */
```

Returns the smallest test-side diagonal element (positive = necessary condition for positive definiteness met). Zero if no diagonal data.

### Multiset Distance

Order-independent distance between two vectors treated as multisets. Counts elements that cannot be matched 1-to-1 within the given tolerance. The result satisfies metric properties: symmetry, non-negativity, and identity of indiscernibles.

For real and integer types, both arrays are sorted and matched via a two-pointer merge. For complex types (C64, C32), a 2D k-d tree is built on one array and nearest-neighbor queries with consumption (used-flags) provide 1-to-1 matching by Euclidean distance. The tolerance threshold is inclusive (less-than-or-equal).

```
int libxs_setdiff(libxs_data_t datatype,
  const void* a, int na,
  const void* b, int nb, double tol);
```

Supports all libxs_data_t types. Returns the number of unmatched elements, or -1 for unsupported types. The distance is at least abs(na - nb) when the vector lengths differ.

```
int libxs_setdiff_min(libxs_data_t datatype,
  const void* a, int na,
  const void* b, int nb, double* tol);
```

Minimizes the multiset distance over all tolerances using Golden Section Search (libxs_gss_min). Returns the minimum unmatched count. The pointer tol (may be NULL) receives the smallest tolerance that achieves this minimum. Supported types: F64, F32, C64, C32 only (integer types are not meaningful here; returns max(na, nb) with tol set to zero).

The distance as a function of tolerance is monotonically non-increasing (a step function with discrete drops), which makes it unimodal — the prerequisite for Golden Section Search.

## Foeppl Polynomial Fingerprint

Structural fingerprint for n-dimensional data based on Foeppl (Macaulay) bracket polynomials. For 1-D data, the fingerprint records per-derivative-order norms of the forward finite differences, normalized to the unit interval [0,1]. Higher dimensions are handled hierarchically: the innermost dimension is fingerprinted first, each child fingerprint is collapsed to a Sobolev self-norm scalar, and those scalars form the 1-D input for the next outer dimension.

Because the fingerprint is normalized to the unit interval, datasets of different lengths (or shapes) produce directly comparable fingerprints. The derivative-order decomposition captures structural features at multiple scales: order 0 measures value magnitude, order 1 measures slope/trend, order 2 measures curvature, and so on.

```
#define LIBXS_FPRINT_MAXORDER 8

typedef struct libxs_fprint_t {
  double l2[LIBXS_FPRINT_MAXORDER + 1];
  double l1[LIBXS_FPRINT_MAXORDER + 1];
  double linf[LIBXS_FPRINT_MAXORDER + 1];
  double mean[LIBXS_FPRINT_MAXORDER + 1];
  double acc_sq[LIBXS_FPRINT_MAXORDER + 1];
  double acc_abs[LIBXS_FPRINT_MAXORDER + 1];
  double acc_sum[LIBXS_FPRINT_MAXORDER + 1];
  double tail[LIBXS_FPRINT_MAXORDER + 1];
  int order, n;
  int nk[LIBXS_FPRINT_MAXORDER + 1];
  libxs_data_t datatype;
} libxs_fprint_t;
```

Three norm families and a signed mean are computed per derivative order k = 0..order:

| Field | Description |
|---|---|
| `l2[k]` | L2 norm of the k-th finite difference |
| `l1[k]` | Mean absolute value (total variation) |
| `linf[k]` | Maximum absolute value (worst-case decay) |
| `mean[k]` | Signed mean (sum / count, preserves sign/phase) |
| `acc_sq[k]` | Accumulator: sum of raw squared differences |
| `acc_abs[k]` | Accumulator: sum of raw absolute differences |
| `acc_sum[k]` | Accumulator: sum of raw signed differences |
| `tail[k]` | Last value at derivative level k (for partial) |
| `nk[k]` | Count of differences accumulated at order k |

All norms are normalized to the unit interval (h = 1/(n-1)). The signed mean preserves the dominant direction of the k-th derivative and breaks pseudometric collisions that pure norms cannot distinguish (negation, reflection).

The `order` field records how many derivative orders were used; `n` is the extent of the fingerprinted dimension. The `datatype` field records the element type — either the type passed by the caller, or the type discovered by probing when the caller passes LIBXS_DATATYPE_UNKNOWN (see "Type Discovery" below).

To recover raw (unnormalized) finite-difference magnitudes, use libxs_fprint_raw (see Generalized Binomial and Distance section).

```
double libxs_fprint_decay(const libxs_fprint_t* info);
```

Geometric-mean per-order decay ratio:

```
r = (l2[K] / l2[0])^(1/K) / (n - 1)
```

The 1/(n-1) factor removes the unit-interval scaling so that decay ratios are comparable across different element counts (without it, fewer elements always appears to have lower decay).

Interpretation of the return value:

```
r << 1    Structured data: forward differences shrink with
          derivative order. Data is smooth and compressible
          under Newton truncation.

r ~= 1    Noise / random data: forward differences grow at the
          rate expected for uncorrelated sequences (~2x per
          order before normalization).

r >> 1    Should not occur for well-formed data; indicates
          divergent finite differences (numerical overflow or
          pathological input).

1e30      Returned when no valid ratio can be computed (e.g.,
          order == 0 or l2[0] == 0 or n < 2).
```

The decay ratio is the key discriminator for type discovery (see below) and for the compressibility diagnostic described in the Foeppl paper.

The L2 norms and signed means serve comparison via the Sobolev distance (libxs_fprint_diff). The Linf norms serve as a decay diagnostic: decaying linf[k] indicates structurally smooth data (compressible under Newton truncation), while growing linf[k] indicates unstructured data (no exploitable smoothness). The L1 norms provide a noise-robust middle ground between L2 and Linf.

```
typedef enum libxs_fprint_flags_t {
  LIBXS_FPRINT_DEFAULT  = 0,
  LIBXS_FPRINT_SORT     = 1,
  LIBXS_FPRINT_AUTOCORR = 2,
  LIBXS_FPRINT_PERAXIS  = 4
} libxs_fprint_flags_t;

int libxs_fprint(libxs_fprint_t* info,
  libxs_data_t datatype, const void* data,
  int ndims, const size_t shape[], const size_t stride[],
  int order, int axis, int smooth, unsigned int flags);
```

Build a fingerprint from data described by shape and stride arrays. For 1-D data, pass ndims=1, shape={n}, stride=NULL. For higher dimensions, shape[0] is the innermost extent and shape[ndims-1] the outermost. When stride is NULL, contiguous storage is assumed (stride[0]=1, stride[k]=product of shape[0..k-1]). The requested order is clamped to min(order, extent-1, LIBXS_FPRINT_MAXORDER).

The flags parameter is a bitwise OR of preprocessing options:

LIBXS_FPRINT_SORT Sort values before differencing (permutation-invariant fingerprint).

LIBXS_FPRINT_AUTOCORR Replace signal with its autocorrelation before differencing (shift-invariant).

LIBXS_FPRINT_PERAXIS Per-axis mode. Differentiates along dimension 'axis' only and takes the per-order maximum of each norm across all positions in the remaining dimensions.

Without LIBXS_FPRINT_PERAXIS, hierarchical mode is used (sweeps the innermost dimension first, collapses via Sobolev self-norms). The axis parameter is only consulted when LIBXS_FPRINT_PERAXIS is set.

The smooth parameter specifies a box-filter radius applied before differencing (0 = off). Each output sample becomes the average of the 2*smooth+1 nearest input samples (clamped at boundaries).

For 1-D data without LIBXS_FPRINT_PERAXIS, axis is ignored.

Supported types: F64, F32, and all integer libxs_data_t types (I64, I32, U32, I16, U16, I8, U8 — promoted to double internally).

When datatype is LIBXS_DATATYPE_UNKNOWN, the function performs automatic type discovery (Level 0 of the hierarchical analysis described below). In this mode, shape[0] is the byte count of the opaque buffer. The function probes all concrete types whose element width divides the byte count, fingerprints each, and selects the interpretation with the smallest decay ratio (libxs_fprint_decay). The discovered type is written to info->datatype. If no candidate has r < 1, the function returns EXIT_FAILURE. Ties are broken by preferring the smallest element width. Float candidates whose maximum absolute value is below 1e-37 (subnormals) are rejected.

```
int libxs_fprint_partial(libxs_fprint_t* info,
  libxs_data_t datatype, const void* data, int n, int order);
```

Streaming (incremental) fingerprint accumulation. Processes data one chunk at a time and maintains exact junction derivatives across calls via the tail buffer. First call: pass a zero-initialized info. Subsequent calls: pass the same info with the next contiguous chunk. Only 1-D contiguous data is supported. The normalized fields (l2, l1, linf, mean) are updated after each call so that libxs_fprint_diff can be used at any intermediate point.

For a single chunk of n elements, produces the same result as libxs_fprint with ndims=1, shape={n}, stride=NULL.

```
int libxs_fprint_join(libxs_fprint_t* output,
  const libxs_fprint_t* a, const libxs_fprint_t* b);
```

Approximate merge of two finalized fingerprints. Combines a and b into output using the raw accumulators (acc_sq, acc_abs, acc_sum). Junction derivatives between the two regions are not recomputed, so the result is approximate with error proportional to order / min(a->n, b->n). The linf field is the maximum of the un-scaled maxima from both sides, re-scaled to the combined length. Returns EXIT_FAILURE if either input has n < 1 or combined n < 2.

```
double libxs_fprint_diff(
  const libxs_fprint_t* a, const libxs_fprint_t* b,
  const double* weights);
```

Weighted Sobolev distance between two fingerprints: d = sqrt( sum over k of weights[k] * [(a->l2[k] - b->l2[k])^2

- (a->mean[k] - b->mean[k])^2] ). The number of orders compared is min(a->order, b->order) + 1. If weights is NULL, default weights w(k) = 1/k! are used, which naturally dampens higher-order (noisier) derivatives.

The inclusion of the signed mean difference breaks collisions from negation (identical L2 norms but opposite means) and reflection (identical L2 but opposite first-derivative means). The distance is a metric in fingerprint space: symmetric, non-negative, zero if and only if the fingerprints are identical. The same function serves both as a distance measure and as a fingerprint comparator since the fingerprint is the decomposed form of the Sobolev norm.

### Type Discovery for Opaque Data

When data arrives as a raw byte stream with no type metadata, the LIBXS toolkit can be composed into a hierarchical analysis that discovers the element type and structure. Each level uses a different tool, exploiting the fact that each tool is sensitive to a different axis of structure while being invariant to others.

```
Tool                Invariant to          Sensitive to
--------------------------------------------------------------
fprint (decay)      element count         consecutive-element
                    and scaling           correlation (local)
setdiff             element order         value identity (global)
matdiff             (positional)          magnitude, spread,
                                          extrema
sort_smooth         row order             row-to-row proximity
shuffle             ---                    provides controlled
                                          reordering
```

The hierarchical procedure:

```
Level 0 --- Decay Screening (libxs_fprint + libxs_fprint_decay)

  Interpret the byte stream under every candidate type whose
  element width divides the stream length. Compute the decay
  ratio r for each. Discard r >= 1 (noise). Short-list the
  best few candidates (within a factor 2 of the minimum r).

  This level is built into libxs_fprint when the caller passes
  LIBXS_DATATYPE_UNKNOWN. The discovered type is reported in
  the datatype field of the returned libxs_fprint_t.

Level 1 --- Self-Consistency via Setdiff (libxs_setdiff_min)

  Split the stream into two halves (A, B) under each surviving
  candidate type. Compute d* = libxs_setdiff_min(A, B). The
  ratio d* / max(|A|, |B|) is a "novelty fraction": how many
  values in one half have no counterpart in the other. A correct
  type produces low novelty (values repeat or cluster); a wrong
  type scatters values across the range (high novelty).

  Synergy: fprint measures local correlation (adjacent elements);
  setdiff measures global value identity (ignoring order). A
  candidate that passes both has independent evidence on two
  orthogonal axes.

Level 2 --- Smoothness-Sort Separability (libxs_sort_smooth)

  Reshape the stream as a matrix under each surviving candidate.
  Compute a GREEDY row permutation that minimizes row-to-row
  Euclidean distance. Fingerprint the permuted matrix along the
  row axis and measure the improvement:

      delta = r_before - r_after

  The candidate with the largest delta has the most exploitable
  row structure.

  Synergy: the GREEDY sort is O(m^2 n) --- too expensive for the
  initial sweep but affordable for 2-3 survivors. Its sensitivity
  (row-to-row proximity) is orthogonal to 1-D decay and to
  order-independent setdiff.

Level 3 --- Shuffle Stability (libxs_shuffle + libxs_fprint_decay)

  Shuffle the interpreted elements via libxs_shuffle (coprime
  permutation). Fingerprint the shuffled data. If the decay
  ratio increases substantially, the original element order
  carries genuine structure. If it stays the same, the apparent
  decay was accidental.

  Synergy: this is the only test that probes whether the ORDER
  of elements matters. Decay (Level 0) measures local correlation
  but cannot distinguish true order from coincidence. Setdiff
  (Level 1) is explicitly order-invariant. The shuffle test
  closes this gap.

Level 4 --- Statistical Plausibility (libxs_matdiff)

  Compute single-matrix statistics (ref=NULL) for each surviving
  candidate: mean, variance, min, max, diagonal range. Reject
  float interpretations with NaN, infinity, or subnormals.
  Flag integer interpretations whose value range spans less than
  1% of the type range. Compare two candidates via
  libxs_matdiff_combine (mean-shift and variance bound).

Stride Sweep --- Record Layout Discovery

  After discovering the atomic element width w, sweep candidate
  strides s that are multiples of w. Reshape the stream as an
  (L/s) x (s/w) matrix. Repeat Levels 0-2 on the outer
  dimension (across "records"). The stride with the smallest
```

```
  decay and largest sort-improvement likely corresponds to the
  record boundary.
```

Why the composition is stronger than any single tool:

- Decay alone is fooled by reinterpretation artifacts (e.g., integer data producing subnormal floats that look smooth).
- Setdiff alone cannot detect structure — it only counts matches, and with large enough tolerance any data matches.
- Sort-smooth alone is too expensive for a brute-force sweep and cannot discriminate types without a baseline decay.
- Shuffle stability alone cannot distinguish true order from accidental order in short sequences.
- Statistics alone reject only pathological cases (NaN, subnormals) and cannot resolve two plausible interpretations.

Each level eliminates a different class of false positives. A candidate that survives all five has demonstrated structure along five orthogonal axes: local smoothness, global value consistency, row separability, order sensitivity, and statistical plausibility. A false positive would require a simultaneous coincidence on all five, which is combinatorially unlikely.

Limitations: the analysis tests fixed-width candidates and cannot discover variable-length encodings, compression, or encryption. For data that is genuinely random at every granularity, all candidates are rejected at Level 0 and the framework reports "no structure found." The stride sweep discovers fixed-size records but not nested or recursive structures.

### Golden Section Search

```
double libxs_gss_min(
  double (*fn)(double x, const void* data),
  const void* data,
  double x0, double x1, double* xmin, int maxiter,
  int flags, double ftol,
  libxs_gss_info_t* info);
```

Minimizes a unimodal function fn on the interval [x0, x1]. The callback receives x and an opaque context pointer. Returns f(x*) where x* is the minimizer; xmin (may be NULL) receives x*.

The bracket shrinks by factor phi = (sqrt(5)-1)/2 per iteration, reusing one evaluation from the previous step. Convergence is reached when the bracket collapses to machine precision or maxiter iterations are exhausted. Flags can enable endpoint evaluation. For step-like objectives such as tolerance thresholds, where plain GSS may discard the side containing the first point of the minimum plateau, use libxs_bisect_min directly.

The optional info pointer (may be NULL) receives diagnostics: status flags, iteration and evaluation counts, the final x/f pair, the final bracket, and a unimodality score in [0, 1]. The score indicates how consistent the sampled points are with a single valley: 1.0 means all samples are monotonically decreasing before the minimum and increasing after; lower values indicate irregularity or multimodality. Endpoint results are reported through status flags rather than through the unimodality score.

```
double libxs_bisect_min(
  double (*fn)(double x, const void* data),
  const void* data,
  double x0, double x1, double fmin, double* xmin, int maxiter,
  double ftol, libxs_gss_info_t* info);
```

Bisects the interval [x0, x1] to find the left edge of a known minimum level fmin. The right endpoint is expected to reach fmin; otherwise info->status includes LIBXS_GSS_STATUS_NO_BRACKET. The function is useful as a standalone primitive for monotone threshold or plateau-edge searches.

### Generalized Binomial and Distance

```
double libxs_binom(double t, int k);
```

Generalized binomial coefficient C(t, k) for real-valued t: C(t, k) = t * (t-1) * ... * (t-k+1) / k!. Evaluates the Newton basis polynomial at position t. For integer t >= k >= 0, this equals the standard binomial coefficient.

```
double libxs_dist2(const double* a, const double* b, int n);
```

Squared Euclidean distance between two n-dimensional vectors. Returns the sum of squared component differences.

```
double libxs_fprint_raw(const libxs_fprint_t* info,
  int k, double value);
```

Recover unnormalized finite-difference magnitude by dividing by (n-1)^k, undoing the unit-interval scaling. For k == 0, the value is returned unchanged.

### Number Theory

```
size_t libxs_gcd(size_t a, size_t b);
size_t libxs_lcm(size_t a, size_t b);
```

GCD (returns 1 for gcd(0,0)) and LCM.

```
int libxs_primes_u32(unsigned int num,
  unsigned int num_factors_n32[], int num_factors_max);
```

Prime factors of `num` in ascending order. Returns factor count.

```
size_t libxs_coprime(size_t n, size_t minco);
size_t libxs_coprime_bias(size_t n, double bias);
size_t libxs_coprime2(size_t n); /* inline: coprime_bias(n, 0) */
```

`libxs_coprime`: co-prime of `n` not exceeding `minco`.

`libxs_coprime_bias`: co-prime of `n` selected by bias in [-1, +1] via a logarithmic mapping (target = n^((1+bias)/2)). bias=-1 selects the smallest non-trivial coprime (maximum displacement), bias=0 selects near sqrt(n) (balanced, same as libxs_coprime2), bias=+1 selects near n/2 (near-alternation). Monotonic: larger bias always yields a larger (or equal) coprime.

`libxs_coprime2`: inline convenience for `libxs_coprime_bias(n, 0)`.

```
unsigned int libxs_remainder(unsigned int a, unsigned int b,
  const unsigned int* limit, const unsigned int* remainder);
```

Smallest multiple of `b` minimizing remainder mod `a`.

```
unsigned int libxs_product_limit(unsigned int product,
  unsigned int limit, int is_lower);
```

Sub-product of prime factors within `limit` (0/1-Knapsack).

### Scalar Utilities

```
double libxs_kahan_sum(double value, double* accumulator,
  double* compensation);
```

Kahan compensated summation.

```
unsigned int libxs_isqrt_u64(unsigned long long x);
unsigned int libxs_isqrt_u32(unsigned int x);
unsigned int libxs_isqrt2_u32(unsigned int x);
```

Integer square root. The isqrt2 variant returns a factor of x.

```
double libxs_pow2(int n);
```

2^n via IEEE-754 exponent. Valid for n in [-1022, 1023]. Returns 0.0 for underflow (subnormals flushed to zero) and +Inf for overflow.

### Modular Arithmetic

```
unsigned int libxs_mod_inverse_u32(unsigned int a, unsigned int m);
size_t libxs_mod_inverse(size_t a, size_t m);
```

Modular inverse via extended Euclidean algorithm: $a^{\{-1\}}$ mod m. Requires gcd(a, m) = 1 and 0 < a, 1 < m. The size_t variant supports cou
- 1.

```
unsigned int libxs_barrett_rcp(unsigned int p);
unsigned int libxs_barrett_pow18(unsigned int p);
unsigned int libxs_barrett_pow36(unsigned int p);
```

Barrett reduction constants for modulus `p`.

```
unsigned int libxs_mod_u32(uint32_t x, unsigned int p,
  unsigned int rcp);                          /* inline */
unsigned int libxs_mod_u64(uint64_t x, unsigned int p,
  unsigned int rcp, unsigned int pow18,
  unsigned int pow36);                        /* inline */
```

Fast modular reduction via Barrett. The 64-bit variant uses a radix-2^18 split.

### BF16 Conversion

```
typedef uint16_t libxs_bf16_t;
libxs_bf16_t libxs_round_bf16_f32(float x); /* inline */
float libxs_bf16_to_f32(libxs_bf16_t v);    /* inline */
libxs_bf16_t libxs_round_bf16(double x);   /* inline */
double libxs_bf16_to_f64(libxs_bf16_t v);  /* inline */
```

Round to BF16 (round-to-nearest-even) and expand to float or double. Uses native `__bf16` when available (`LIBXS_BF16`), otherwise portable bit manipulation.

```
void libxs_dekker_bf16(double val, int ndigits,
  libxs_bf16_t* dst);                       /* inline */
void libxs_dekker_bf16_f32(float val, int ndigits,
  libxs_bf16_t* dst);                       /* inline */
```

Dekker split: decompose a value into `ndigits` BF16 residuals via iterative subtraction. `dst[0]` holds the dominant digit (the BF16 rounding of the input); `dst[1]` holds the residual after subtracting the first digit's contribution, rounded to BF16; and so on. The sum `libxs_bf16_to_f64(dst[0]) + ... + libxs_bf16_to_f64(dst[ndigits-1])` reconstructs the original to approximately `7 * ndigits` bits of precision (each BF16 digit carries 7 fraction bits). Seven digits exceed the 52-bit mantissa of double. The `_f32` variant keeps the residual arithmetic in single precision for float storage/conversion paths.

## Memory Operations

Header: `libxs_mem.h`

Byte-level memory macros, pointer alignment queries, buffer comparison, and matrix copy/transposition.

### Memory Macros

```
LIBXS_MEMSET(DST, SRC, SIZE)
```

Set SIZE bytes at DST to the value SRC, unrolled at compile-time.

```
LIBXS_MEMZERO(DST)
```

Zero all bytes of *DST (size derived from sizeof(*DST)).

```
LIBXS_MEMCPY(DST, SRC, SIZE)
```

Copy SIZE bytes from SRC to DST, unrolled at compile time.

```
LIBXS_ASSIGN(DST, SRC)
```

Copy sizeof(*SRC) bytes from SRC to DST. Asserts sizeof(*SRC) <= sizeof(*DST).

```
LIBXS_MEMSWP(PTR_A, PTR_B, SIZE)
```

Swap SIZE bytes between PTR_A and PTR_B. Asserts PTR_A != PTR_B.

```
LIBXS_VALUE_ASSIGN(DST, SRC)
LIBXS_VALUE_SWAP(A, B)
```

Operate on L-values directly (address-of is taken internally). VALUE_ASSIGN can cast away const qualifiers. VALUE_SWAP asserts equal size.

### Offset and Alignment

```
size_t libxs_offset(size_t ndims, const size_t offset[],
  const size_t shape[], size_t* size);
```

Compute the linear offset of an n-dimensional coordinate within a shape. Optionally returns the total linear size of the shape.

```
int libxs_aligned(const void* ptr, const size_t* inc,
  int* alignment);
```

Check whether ptr is SIMD-aligned (optionally considering the next access at ptr + *inc). Optionally writes the pointer alignment in bytes to *alignment.

### Comparison

```
unsigned char libxs_diff(const void* a, const void* b,
  unsigned char size);
```

Compare two short buffers. Returns zero when equal, non-zero otherwise. Uses SSE/AVX2/AVX-512 when available.

```
unsigned int libxs_diff_n(const void* a, const void* bn,
  unsigned char elemsize, unsigned char stride,
  unsigned int hint, unsigned int count);
```

Compare a against count elements in bn. Returns the first index that matches a, or count if none match. hint supplies the initial search position.

```
int libxs_memcmp(const void* a, const void* b, size_t size);
```

Boolean variant of the C memcmp. Returns zero when equal. Uses SSE/AVX2/AVX-512 when available.

### Matrix Copy and Transposition

```
void libxs_matcopy(void* out, const void* in,
  unsigned int typesize, int m, int n, int ldi, int ldo);
void libxs_matcopy_task(void* out, const void* in,
  unsigned int typesize, int m, int n, int ldi, int ldo,
  int tid, int ntasks);
```

Copy an m-by-n matrix from in to out with leading dimensions ldi and ldo. If in is NULL the destination is zeroed. The _task variant distributes work across ntasks threads (caller passes its tid).

```
void libxs_otrans(void* out, const void* in,
  unsigned int typesize, int m, int n, int ldi, int ldo);
void libxs_otrans_task(void* out, const void* in,
  unsigned int typesize, int m, int n, int ldi, int ldo,
  int tid, int ntasks);
```

Out-of-place matrix transposition. The _task variant distributes work across ntasks threads.

```
void libxs_itrans(void* inout, unsigned int typesize,
  int m, int n, int ldi, int ldo, void* scratch);
void libxs_itrans_task(void* inout, unsigned int typesize,
  int m, int n, int ldi, int ldo, void* scratch,
  int tid, int ntasks);
```

In-place matrix transposition (square or via scratch buffer). scratch can be NULL (auto-allocated).

```
void libxs_itrans_batch(void* inout, unsigned int typesize,
  int m, int n, int ldi, int ldo,
  int index_base, int index_stride,
  const int stride[], int batchsize,
  int tid, int ntasks);
```

Batch of in-place matrix transpositions (per-thread form).

## MetaImage I/O

Header: `libxs_mhd.h`

Read and write image data in the MetaImage (MHD/MHA) format. Files are compatible with ITK, ITK-SNAP, 3D Slicer, ParaView, and similar tools.

### Types

```
typedef enum libxs_mhd_element_conversion_hint_t {
  LIBXS_MHD_ELEMENT_CONVERSION_DEFAULT,
  LIBXS_MHD_ELEMENT_CONVERSION_MODULUS
} libxs_mhd_element_conversion_hint_t;
```

Hint controlling element conversion behaviour (default clamping vs. modulus mapping).

```
typedef struct libxs_mhd_element_handler_info_t {
  libxs_data_t type;
  libxs_mhd_element_conversion_hint_t hint;
} libxs_mhd_element_handler_info_t;
```

Destination type and conversion hint, passed to built-in or custom element handlers.

```
typedef int (*libxs_mhd_element_handler_t)(void* dst,
  const libxs_mhd_element_handler_info_t* dst_info,
  libxs_data_t src_type,
  const void* src, const void* src_min, const void* src_max,
  size_t index, void* context);
```

Per-element callback for reading or converting image data. The value range (`src_min`, `src_max`) is available for scaling. `index` is the logical element index in write/read traversal order, and `context` is user data supplied by the caller.

```
typedef struct libxs_mhd_info_t {
  size_t ndims;          /* dimensionality */
  size_t ncomponents;    /* interleaved image channels */
  libxs_data_t type;     /* pixel element type */
  size_t header_size;    /* header size in bytes (LOCAL data) */
} libxs_mhd_info_t;
```

Image descriptor populated by `libxs_mhd_read_header` and consumed by read/write.

```
typedef struct libxs_mhd_write_info_t {
  const libxs_mhd_element_handler_info_t* handler_info;
  libxs_mhd_element_handler_t handler;
  void* handler_context;
  const char* extension_header;
  const void* extension;
  size_t extension_size;
} libxs_mhd_write_info_t;
```

Optional write parameters (conversion, custom handler, extension data). NULL is accepted for all-defaults.

### Element Helpers

```
int libxs_mhd_element_conversion(void* dst,
  const libxs_mhd_element_handler_info_t* dst_info,
  libxs_data_t src_type,
  const void* src, const void* src_min, const void* src_max,
  size_t index, void* context);
```

Built-in element conversion. Scales values when `src_min` and `src_max` are non-NULL; otherwise clamps to the destination type.

```
int libxs_mhd_element_comparison(void* dst,
  const libxs_mhd_element_handler_info_t* dst_info,
  libxs_data_t src_type,
  const void* src, const void* src_min, const void* src_max,
  size_t index, void* context);
```

Check an in-memory buffer against file content (element-wise comparison with optional type conversion).

### Type Queries

```
const char* libxs_mhd_typename(libxs_data_t type,
  const char** ctypename);
```

Return the MHD element-type name (e.g. `"MET_FLOAT"`) for the given `libxs_data_t`. Optionally writes the C type name. Returns NULL for unknown types.

```
libxs_data_t libxs_mhd_typeinfo(const char elemname[]);
```

Return the `libxs_data_t` for a given MHD element-type string.

### Read

```
int libxs_mhd_read_header(const char header_filename[],
  size_t filename_max_length, char filename[],
  libxs_mhd_info_t* info, size_t size[],
  char extension[], size_t* extension_size);
```

Parse an MHD header file. `info->ndims` is an in/out parameter (maximum on input, actual on output). `filename` receives the data-file path (may differ from the header when data is stored separately). Extension data is optional.

```
int libxs_mhd_read(const char filename[],
  const size_t offset[], const size_t size[],
  const size_t pitch[], const libxs_mhd_info_t* info,
  void* data,
  const libxs_mhd_element_handler_info_t* handler_info,
  libxs_mhd_element_handler_t handler,
  void* handler_context);
```

Read image data into `data`. Optional `handler_info` triggers type conversion; optional `handler` supplies a custom per-element callback (can also serve as a comparison function without allocating a buffer).

### Write

```
int libxs_mhd_write(const char filename[],
  const size_t offset[], const size_t size[],
  const size_t pitch[], libxs_mhd_info_t* info,
  const void* data,
  const libxs_mhd_write_info_t* write_info);
```

Write image data in the extended MHD format. `info->header_size` is updated on output. Pass NULL for `write_info` when no conversion or extension is needed.

When the environment variable `LIBXS_MHD_PNG` is set and the image is two-dimensional, an uncompressed PNG file is written alongside the MHD output (same base name, `.png` extension). Supported channel counts are 1 (grayscale), 3 (RGB), and 4 (RGBA) at 8-bit depth. Source data of any element type is converted to unsigned 8-bit using the built-in element conversion with automatic min/max scaling.

## Mix

Header: `libxs_mix.h`

Fixed-share mixture over a bank of experts: a linear pool whose per-expert weights are carried across observations and updated from realized log loss, so the mixture tracks whichever expert is currently right. Experts may abstain per observation.

Not to be confused with `libxs_predict_set_series_bank`, which averages several views of one window with fixed equal weight; this learns the weights.

## Types

```
typedef struct libxs_mix_t {
  double* weight;  /* per-slot weight [nslot] */
  int nslot;       /* number of experts */
  double rate;     /* exponent on the likelihood ratio */
  double share;    /* uniform share redistributed per step */
  double relmin;   /* floor for a non-positive ratio (0 disables) */
} libxs_mix_t;
```

The struct is plain data. A caller that already owns a weight array may fill the fields directly and point `weight` at it, which suits a bank living on the stack or inside a larger record; such a view must not be passed to `libxs_mix_destroy`.

## Functions

```
int libxs_mix_create(libxs_mix_t* mix, int nslot,
  double rate, double share, double relmin);
void libxs_mix_destroy(libxs_mix_t* mix);
void libxs_mix_reset(libxs_mix_t* mix, const int active[]);
```

`create` starts from a uniform prior and returns `EXIT_SUCCESS` or `EXIT_FAILURE`. `reset` restores a uniform prior over the slots marked active (NULL selects all). `destroy` releases the weights; the struct itself is caller-owned.

```
double libxs_mix_pool(const libxs_mix_t* mix,
  const double prob[], const int active[]);
void libxs_mix_update(libxs_mix_t* mix,
  const double prob[], const int active[], double mixture);
double libxs_mix_observe(libxs_mix_t* mix,
  const double prob[], const int active[]);
```

`prob[k]` is expert `k`'s probability for the outcome under consideration. `active` may be NULL (every expert speaks). `pool` reports the mixture without touching the weights. `update` advances the weights against a mixture the caller supplies. `observe` is `pool` followed by `update` with that pool, and returns the pooled value taken *before* the update.

Prefer `observe`: the pooled value must be committed before the weights move, or a reported code length is better than the truth with no symptom to notice. Use `update` only when the caller's mixture is genuinely not `pool`'s — because it mixes in experts this bank does not carry, or because it was floored before a logarithm was taken.

## Abstention and Disabled Slots

Abstention (`active[k] == 0`) is not a probability of zero. The pool renormalizes over the experts that spoke, so a silent expert neither dilutes the mixture nor loses weight for staying quiet. With every expert active the renormalization is a no-op, because the weights sum to one.

A weight of zero means the slot is disabled permanently: a bank sized for the widest configuration holds unused slots at zero and share mass does not resurrect them. A bank whose slots are a fixed ladder that should always stay revivable is not a client of this module.

`relmin` guards the other direction. Without it an expert that once gave the outcome no mass at all is multiplied by zero and can never recover, because the share term only reaches slots that still hold mass.

### Example

```
libxs_mix_t mix;
double prob[3];
if (EXIT_SUCCESS == libxs_mix_create(&mix, 3, 0.15, 0.005, 1e-4)) {
  double bits = 0;
  while (next_observation(prob)) {
    const double p = libxs_mix_observe(&mix, prob, NULL);
    bits -= log(0 < p ? p : 1e-12) / log(2.0);
  }
  libxs_mix_destroy(&mix);
}
```

## N-gram

Header: `libxs_ngram.h`

Count-based variable-order n-gram model over unsigned token ids, backed by a registry keyed on the context. Stores every context length up to a configured maximum from one pass of observations, then answers interpolated-backoff probabilities and top-k predictions.

Token id 0 is reserved and treated as absent context: it is never stored and never predicted.

### Constants

```
#define LIBXS_NGRAM_ORDER_MAX 6  /* longest context that can be stored */
#define LIBXS_NGRAM_SUCC_MAX  8  /* successors retained per context */
```

### Types

```
typedef struct libxs_ngram_succ_t {
  unsigned int id, count;
} libxs_ngram_succ_t;

typedef struct libxs_ngram_entry_t {
  unsigned int total;   /* observations of this context */
  unsigned int nsucc;   /* successors retained */
  libxs_ngram_succ_t succ[LIBXS_NGRAM_SUCC_MAX];
} libxs_ngram_entry_t;
```

The successor list is bounded by a Space-Saving policy that evicts the least frequent entry, so `total` may exceed the sum of the retained counts. The difference is real uncertainty, and the probability model charges it to backoff rather than hiding it.

```
typedef struct libxs_ngram_t {
  libxs_registry_t* store;
  unsigned int* unifreq;
  double unifreq_total;
  unsigned int unifreq_size;
  unsigned int backoff_ids[LIBXS_NGRAM_SUCC_MAX];
  int backoff_count, maxorder;
} libxs_ngram_t;
```

Callers own the struct, so stack allocation is fine; the store and backoff tables are heap-managed by create and destroy.

```
typedef struct libxs_ngram_stats_t {
  long keys[LIBXS_NGRAM_ORDER_MAX + 1];       /* per order, index 0 unused */
  double obs[LIBXS_NGRAM_ORDER_MAX + 1];
  long saturated[LIBXS_NGRAM_ORDER_MAX + 1];
  size_t entries, nbytes;
} libxs_ngram_stats_t;
```

### Functions

```
int libxs_ngram_create(libxs_ngram_t* model, int maxorder);
void libxs_ngram_destroy(libxs_ngram_t* model);
```

`maxorder` is clamped to `1..LIBXS_NGRAM_ORDER_MAX`. `create` returns `EXIT_SUCCESS`, or `EXIT_FAILURE` if the backing store cannot be created.

```
void libxs_ngram_observe(libxs_ngram_t* model,
  const unsigned int hist[], int hlen, unsigned int succ);
void libxs_ngram_finalize(libxs_ngram_t* model, unsigned int vocab);
```

`observe` increments every context length `1..min(maxorder,hlen)` ending at `hist[hlen-1]`, so one call per position populates all orders. `finalize` builds the global unigram distribution and the most-frequent-token backoff list from the order-1 counts; call it once after all observations and before any query, with `vocab` the largest token id in use.

```
const libxs_ngram_entry_t* libxs_ngram_lookup(const libxs_ngram_t* model,
  const unsigned int hist[], int hlen, int n);
double libxs_ngram_prob(const libxs_ngram_t* model,
  const unsigned int hist[], int hlen, unsigned int next);
int libxs_ngram_predict(const libxs_ngram_t* model,
  const unsigned int hist[], int hlen, unsigned int out_ids[], int k,
  int* order);
int libxs_ngram_stats(const libxs_ngram_t* model,
  libxs_ngram_stats_t* stats);
```

`prob` is interpolated backoff: it blends orders `1..min(maxorder,hlen)` with weight `total/(total+1)` per order over an additive-smoothed unigram prior. After `finalize`, probabilities over ids `1..vocab` sum to one for every history — which makes the model usable as a code-length model and not only as a ranker.

`predict` uses hard backoff instead: the top-k successors of the longest context that has a record, falling back to the global most-frequent list when none matches. A non-NULL `order` receives the context length that produced the result (0 for the global fallback), so a caller can tell a deeply grounded prediction from a guess.

`stats` reports per-order key and observation counts plus the live footprint. Report `entries` as the honest size: `nbytes` follows the hash table's power-of-two capacity and therefore steps rather than growing smoothly.

### Example

```
libxs_ngram_t model;
unsigned int hist[LIBXS_NGRAM_ORDER_MAX];
int hlen = 0;
if (EXIT_SUCCESS == libxs_ngram_create(&model, 3)) {
  unsigned int id;
  while (next_token(&id)) {
    libxs_ngram_observe(&model, hist, hlen, id);
    if (LIBXS_NGRAM_ORDER_MAX > hlen) hist[hlen++] = id;
    else {
      memmove(hist, hist + 1, (LIBXS_NGRAM_ORDER_MAX - 1) * sizeof(*hist));
      hist[LIBXS_NGRAM_ORDER_MAX - 1] = id;
    }
  }
  libxs_ngram_finalize(&model, vocab);
  /* ... libxs_ngram_prob / libxs_ngram_predict ... */
  libxs_ngram_destroy(&model);
}
```

## Permutation and Sorting

Header: `libxs_perm.h`

Permutation-based data reordering: deterministic co-prime shuffling (in-place and out-of-place with optional SIMD gather) and smoothness-optimized row permutations for matrices.

### Shuffling

The shuffle functions use a co-prime stride C to produce a fixed, deterministic permutation of count elements. The mapping is affine:

```
dst[k] = src[(N-1) - ((C*k + offset) mod N)]
```

The stride must be co-prime to count; passing NULL selects libxs_coprime2(count). The offset parameter extends the basic co-prime permutation into an affine family, expanding the number of available permutations from ~phi(N)/2 to ~N*phi(N)/2. Pass offset=0 for the standard (non-affine) shuffle.

```
int libxs_shuffle(void* inout, size_t elemsize, size_t count,
  const size_t* shuffle, size_t offset,
  const size_t* nrepeat);
```

In-place shuffle of count elements of elemsize bytes each. Uses cycle following with a bit vector (N/8 bytes auxiliary). nrepeat controls the number of successive permutation applications (NULL or pointing to 1 means one pass). Returns EXIT_SUCCESS or EXIT_FAILURE.

```
int libxs_shuffle2(void* dst, const void* src, size_t elemsize,
  size_t count, const size_t* shuffle, size_t offset,
  const size_t* nrepeat);
```

Out-of-place shuffle from src to dst. dst and src must not overlap. If *nrepeat is zero an ordinary copy is performed. Uses AVX2/AVX-512 gather instructions when available for 4- and 8-byte elements (offset=0 path).

```
size_t libxs_unshuffle(size_t count, const size_t* shuffle);
```

Return the number of libxs_shuffle2 applications needed to restore the original element order for the given count and co-prime stride. The cycle length depends only on C and N, not on the offset.

```
int libxs_unshuffle2(void* dst, const void* src, size_t elemsize,
  size_t count, const size_t* shuffle, size_t offset,
  const size_t* nrepeat);
```

Single-pass inverse of libxs_shuffle2. Computes the modular inverse C_inv = C^{-1} mod N via the extended Euclidean algorithm, then gathers elements using the inverse permutation:

```
dst[m] = src[C_inv * (N-1-offset-m) mod N]
```

This restores the original order in one O(N) pass rather than the R-1 repeated applications that libxs_unshuffle would require. The loop structure (sequential write, strided read) is identical to the forward shuffle and amenable to the same SIMD gather optimizations. Supports multi-pass inversion (nrepeat > 1) by iterating f^{-1}.

## General-Purpose Sort

```
typedef int (*libxs_sort_cmp_t)(
  const void* a, const void* b, void* ctx);

void libxs_sort(void* base, int n, size_t size,
  libxs_sort_cmp_t cmp, void* ctx);
```

Sort n elements of the given byte size. The comparator returns negative, zero, or positive (tristate, like qsort_r). The ctx pointer is forwarded to the comparator unchanged.

Built-in comparators:

```
int libxs_cmp_f64(const void* a, const void* b, void* ctx);
int libxs_cmp_f32(const void* a, const void* b, void* ctx);
int libxs_cmp_i32(const void* a, const void* b, void* ctx);
int libxs_cmp_u32(const void* a, const void* b, void* ctx);
```

Built-in comparators enable an O(n) radix sort fast path. Custom comparators use O(n log n) heap sort.

With a built-in comparator, ctx controls the mode:

ctx = NULL Sort base in-place. ctx != NULL Read from ctx, write sorted result to base (out-of-place, source unchanged).

With a custom comparator, ctx is user state passed to cmp and base is sorted in-place.

Examples:

libxs_sort(data, n, sizeof(double), libxs_cmp_f64, NULL); libxs_sort(dst, n, sizeof(double), libxs_cmp_f64, src); libxs_sort(perm, n, sizeof(int), my_indirect_cmp, keys);

## Space-Filling Curves

```
uint64_t libxs_hilbert(const unsigned int coords[], int ndims);
void libxs_hilbert_decode(uint64_t code, unsigned int coords[], int ndims);
```

N-dimensional Hilbert curve index. Maps ndims coordinates to a 64-bit key with strong locality guarantees. Each coordinate is quantized to floor(64/ndims) bits. coords[k] must be in [0, 2^bits_per_dim). The decode routine maps such a key back to ndims coordinates with the same bit budget.

```
uint64_t libxs_morton(const unsigned int coords[], int ndims);
void libxs_morton_decode(uint64_t code, unsigned int coords[], int ndims);
```

N-dimensional Morton code (Z-order curve). Bit-interleaves ndims coordinates into a 64-bit key. Each coordinate is quantized to floor(64/ndims) bits. The decode routine deinterleaves the key back to ndims coordinates.

```
int libxs_stratify_morton(
  const unsigned int src_coords[], int src_ndims,
  unsigned int dst_coords[], int dst_ndims);
int libxs_stratify_hilbert(
  const unsigned int src_coords[], int src_ndims,
  unsigned int dst_coords[], int dst_ndims);
```

Stratify higher-dimensional coordinates into a lower-dimensional layout without slicing. The source coordinates are encoded using the selected source-dimensional space-filling curve, and the resulting key is decoded as target-dimensional coordinates using the same curve family. This gives deterministic rank-preserving embeddings such as 3D to 2D:

```
code = curve(src_coords, 3)
dst_coords = inverse_curve(code, 2)
```

The target dimensionality must be smaller than the source dimensionality. The functions return EXIT_SUCCESS or EXIT_FAILURE.

### k-d Tree

```
typedef int (*libxs_kdtree_split_t)(
  int* dim, int* pos, const double* pts, int* idx,
  int count, int depth, int nleaves, void* ctx);

typedef struct libxs_kdtree_config_t {
  int min_leaf;
  libxs_kdtree_split_t split;
  void* ctx;
} libxs_kdtree_config_t;
```

Split callback for custom tree construction. Writes chosen split dimension into *dim and left-child count into *pos. The callback may reorder idx[0..count-1] to place left entries first; if it does, the partition function skips its internal quickselect. nleaves gives the current number of leaves already assigned. Return 0 for a valid split, nonzero to force a leaf.

Config struct: min_leaf controls minimum node size (0 = split down to 1 element). split=NULL uses round-robin median splits.

```
void libxs_kdtree_build(const double* pts, int* idx, int n,
  int ndims, int stride, const libxs_kdtree_config_t* config);
```

Build a k-d tree in-place. Points are stored row-major: pts[i*stride + k] is coordinate k of point i. The index array idx[0..n-1] is rearranged into implicit tree structure. config may be NULL (round-robin median, leaf at 1 element).

```
int libxs_kdtree_partition(const double* pts, int* idx, int n,
  int ndims, int stride, int* assignments,
  const libxs_kdtree_config_t* config);
```

Partition n points into leaves and write leaf IDs into assignments[0..n-1]. Returns the number of leaves. Same tree logic as build, but produces cluster assignments instead of a query-tree index.

```
int libxs_kdtree_nearest(const double* pts, const int* idx,
  const unsigned char* used, int n, int ndims, int stride,
  const double* query, double max_dist2);
```

Find the nearest point to query[0..ndims-1] within squared Euclidean distance max_dist2. Returns the point index or -1. The used array (may be NULL) marks consumed points.

```
int libxs_kdtree_knearest(const double* pts, const int* idx,
  const unsigned char* used, int n, int ndims, int stride,
  const double* query, double max_dist2,
  int out_idx[], double out_dist2[], int k);
```

Find the k nearest points, writing indices to out_idx and their squared distances to out_dist2, both sorted ascending, and returning how many were written (0..k; fewer when n or max_dist2 bounds it).

The result is exact: the same k points a linear scan would pick, in the same order. The tree only skips subtrees that cannot hold a closer point, so it replaces a scan without changing the answer — which matters when the neighbours feed an estimate rather than only a ranking.

Both searches require a tree built with `config == NULL` (or a config whose `split` is NULL), because the query recomputes the split dimension as `depth % ndims`. A tree built with a custom split function does not record its choices and cannot be searched.

A kd-tree stops paying as dimension rises: it beats a scan while n is large against 2^ndims, and degenerates toward visiting every node beyond roughly ten dimensions. Prefer a scan there.

Convenience wrappers for 2D (interleaved x,y layout):

```
void libxs_kdtree2d_build(double* pts, int* idx, int n);
int libxs_kdtree2d_nearest(const double* pts, const int* idx,
  const unsigned char* used, int n,
  double x, double y, double max_dist2);
```

### Smooth Sorting

```
typedef enum libxs_sort_t {
  LIBXS_SORT_NONE     = 0,
  LIBXS_SORT_IDENTITY = 1,
  LIBXS_SORT_NORM     = 2,
  LIBXS_SORT_MEAN     = 3,
  LIBXS_SORT_GREEDY   = 4,
  LIBXS_SORT_MORTON   = 5,
  LIBXS_SORT_HILBERT  = 6
} libxs_sort_t;

int libxs_sort_smooth(libxs_sort_t method, int m, int n,
  const void* mat, int ld, libxs_data_t datatype, int* perm);
```

Compute a row permutation that reorders the rows of an m-by-n column-major matrix for smoothness (decaying forward differences between consecutive rows).

Parameters:

method Sorting strategy (see constants above). m, n Matrix dimensions (m rows, n columns). mat Column-major matrix data (read-only). ld Leading dimension (ld >= m). datatype Element type (F64, F32, I32, U32, I16, U16, I8, U8). perm Output array of m ints; perm[i] = source row for position i after reordering.

Methods:

NONE No permutation (early return). IDENTITY Identity permutation (perm[i] = i). NORM Sort rows by ascending L1-norm. MEAN Sort rows by ascending column mean. GREEDY Nearest-neighbor chain: starting from row 0, each step picks the unvisited row with the smallest Euclidean distance to the current row. MORTON Sort rows by Morton (Z-order) key computed from quantized column values. HILBERT Sort rows by Hilbert curve key computed from quantized column values.

Returns EXIT_SUCCESS or EXIT_FAILURE.

### Parameter Prediction

Start Presentation

Header: `libxs_predict.h`

Predict output parameters from input parameters using fingerprint-guided polynomial interpolation and kNN classification. Supports incremental training, model persistence, confidence-gated evaluation, output transforms, and thread-safe operation.

### Mode Flags

```
typedef enum libxs_predict_mode_t {
  LIBXS_PREDICT_AUTO        = 0,
  LIBXS_PREDICT_INTERPOLATE = 1,
  LIBXS_PREDICT_CLASSIFY    = 2,
  LIBXS_PREDICT_TEMPORAL    = 4
} libxs_predict_mode_t;
```

ORable flags controlling prediction behavior:

- AUTO (0): fingerprint decides per-output (default).
- INTERPOLATE: force polynomial for all outputs.
- CLASSIFY: force kNN vote for all outputs.

- TEMPORAL: timeseries mode — enables recency weighting (recent neighbors preferred), continuous output (no snap-to-nearest), and local coherence smoothing across horizon steps. Only effective when set_series was called (nseries > 0). For timeseries models without this flag, temporal heuristics auto-enable only when the query falls outside the training bounding box.

Example: `LIBXS_PREDICT_CLASSIFY | LIBXS_PREDICT_TEMPORAL` forces kNN-weighted projection forward with temporal heuristics.

#### Input Decomposition and Feature Selection

```
typedef enum libxs_predict_decompose_t {
  LIBXS_PREDICT_RAW     = 0,
  LIBXS_PREDICT_SPREAD  = 1,
  LIBXS_PREDICT_PCA     = 2,
  LIBXS_PREDICT_SETDIFF = 3,
  LIBXS_PREDICT_FISHER  = 4,
  LIBXS_PREDICT_RF      = 5,
  LIBXS_PREDICT_HKNN    = 6
} libxs_predict_decompose_t;
```

Controls input processing and prediction strategy:

- RAW (0): no decomposition, standard kNN (default).
- SPREAD: sum/diff modes for anti-correlated pairs (2 series). Forward: sum = (A+B)/2, diff = (A-B)/2. Inverse: A = sum+diff, B = sum-diff. Only effective when nseries == 2.
- PCA: principal component rotation of the input space. At build time, computes eigenvectors of the input covariance matrix (Jacobi iteration), stores the rotation matrix, and transforms all entries into PC space. Weights are auto-set to zero out PCs below the 95% cumulative variance threshold (dimensionality reduction). Applied via libxs_gemm.
- SETDIFF: automatic feature selection using libxs_setdiff. At build time, scores each input dimension by class separability (per-class-pair setdiff with 5% tolerance, range-normalized). Zeroes weights for features below the median score. Supervised (uses output labels).
- FISHER: automatic feature selection using Fisher's discriminant ratio (between-class / within-class variance). At build time, computes per-feature Fisher score, applies sqrt weighting above median, zeroes below. Supervised. Generally best for kNN classification.
- RF: Random Forest classification. At build time, trains an ensemble of 100 decision trees per output (bootstrap sampling, sqrt(ninputs) random features per split, Gini impurity). At eval time, returns majority vote across trees. Per-output confidence = vote fraction. Excels on high-dimensional classification (37+ features). Persisted in save/load (compact on-disk format: 15 bytes/node).
- HKNN: hierarchical kNN. At build time, partitions data using a Fisher-guided kd-tree (output-aware dimension selection, adaptive balance penalty) followed by Lloyd refinement. Uses Gini impurity when a single categorical output is detected. A soft cap based on the Fisher/Gini score suppresses marginal splits near target cluster count. Inference uses standard kNN within each partition. Best for structured parameter prediction where output-aware spatial partitioning outperforms geometric k-means.

PCA/SPREAD decomposition is applied at build time to stored entries and at eval time to user queries. Inverse prediction returns inputs in raw (user) space. Feature selection modes (SETDIFF, FISHER) only set weights at build time. RF replaces the kNN eval path entirely. HKNN replaces only the clustering step (kNN inference is unchanged).

#### Output Transforms

```
typedef enum libxs_predict_transform_t {
  LIBXS_PREDICT_IDENTITY = 0,
  LIBXS_PREDICT_LOG      = 1,
  LIBXS_PREDICT_SQRT     = 2
} libxs_predict_transform_t;
```

Per-output transforms applied transparently:

- IDENTITY (0): no transform (default).
- LOG: forward = log(x+1), inverse = exp(x)-1.
- SQRT: forward = sqrt(x), inverse = x*x.

The forward transform is applied during push, the inverse during eval. The fingerprint and kNN operate in transformed space where heavy-tailed data is smoother.

### Create and Destroy

```
libxs_predict_t* libxs_predict_create(int ninputs, int noutputs);
void libxs_predict_destroy(libxs_predict_t* model);
libxs_lock_t* libxs_predict_lock(libxs_predict_t* model);
```

Create a model for the given input/output dimensionality. Returns NULL on invalid arguments. Destroy accepts NULL. The lock accessor returns a pointer to the model's internal lock for use with push/eval (NULL if model is NULL).

### Configuration

```
void libxs_predict_set_mode(libxs_predict_t* model, int mode);
```

Set prediction mode (ORable flags from libxs_predict_mode_t). Default is LIBXS_PREDICT_AUTO.

```
void libxs_predict_set_weights(libxs_predict_t* model,
  const double weights[]);
```

Set per-dimension input weights for distance computation. Larger weight means the dimension contributes more. Pass NULL to reset to uniform weighting. Must be called before libxs_predict_build.

```
void libxs_predict_set_transform(libxs_predict_t* model,
  int output, int transform);
```

Set per-output transform. The output parameter is 0-based, or -1 to set all outputs. Must be called before pushing entries. Transforms are persisted in save/load.

```
void libxs_predict_set_refine(libxs_predict_t* model,
  int iterations);
```

Set the number of forward-inverse-forward refinement iterations applied during eval. Default (0): iterate automatically when per-output confidence drops below 0.9. When set to >0, always perform the given number of iterations regardless of confidence.

```
void libxs_predict_set_smooth(libxs_predict_t* model,
  double amount);
```

Control multi-cluster blending at eval time.

- amount<0: auto-determine blending radius from model structure.
- amount=0: no blending (default after manual assignment).
- amount>0: blend predictions from clusters within (1+amount)*nearest_distance radius. Only smooth-mode outputs are blended; categorical outputs remain unblended.

```
void libxs_predict_set_consistency(libxs_predict_t* model,
  double amount);
```

Set round-trip consistency penalty (0..1). During the refinement loop, the prediction is inverse-mapped back to input space. If the reconstructed input differs from the original query by more than the cluster diameter, the prediction is geometrically inconsistent.

- amount=0 (default): inconsistency only skips refinement (no confidence change).
- amount>0: confidence is actively penalized proportional to the round-trip distance: `conf = quality + conf / (1 + amount`
- amount=1: halves confidence above the quality floor when the round-trip distance equals the cluster diameter.

This catches predictions where the model contradicts itself: the output maps back to a different input region than the query came from.

```
void libxs_predict_set_quantile(libxs_predict_t* model,
  double quantile);
```

Set quantile level for prediction intervals (0..0.5). When enabled, `info->lower` and `info->upper` are filled with the q-th and (1-q)-th distance-weighted quantiles of the k nearest neighbors.

- quantile=0 (default): intervals not computed, `info->lower` and `info->upper` are NULL.
- quantile>0 (e.g. 0.1): 10th/90th percentile bounds, scaled by `1/confidence` so sparse regions produce wider intervals and dense regions stay close to the raw neighbor spread.

The interval captures "plausible range given the neighbors" while confidence captures "should I trust this at all." Narrow interval + high confidence = strong prediction.

### Timeseries Configuration

```
void libxs_predict_set_series(libxs_predict_t* model,
  int nseries, int window);
void libxs_predict_set_target(libxs_predict_t* model,
  int target);
void libxs_predict_set_decompose(libxs_predict_t* model,
  int decompose);
void libxs_predict_set_diff(libxs_predict_t* model,
  int order);
```

Declare timeseries structure for automatic sliding-window construction. The model's ninputs must equal nseries * window; noutputs is the forecast horizon.

When set_series has been called, push(lock, model, values, NULL) accumulates one timestep (nseries values per call). The build step then constructs all valid sliding windows internally: input = concatenated windows across all series, output = the next horizon values of the target series.

set_target selects which series to predict (0-based, default 0). set_decompose selects input processing mode (see above).

set_diff enables auto-differencing for non-stationary series:

- `set_diff(model, 0)`: auto-detect order from fingerprint decay.
- `set_diff(model, d)`: explicit order d (1 = linear detrend).
- Default is disabled (-1).

Single-series example (sunspots):

```
model = libxs_predict_create(WINDOW, HORIZON);
libxs_predict_set_series(model, 1, WINDOW);
for (t = 0; t < n; ++t)
    libxs_predict_push(NULL, model, &series[t], NULL);
libxs_predict_build(model, 0, 2, 0);
```

Multi-series example (anti-correlated pair):

```
model = libxs_predict_create(WINDOW * 2, HORIZON);
libxs_predict_set_series(model, 2, WINDOW);
libxs_predict_set_target(model, 0);
libxs_predict_set_decompose(model, LIBXS_PREDICT_SPREAD);
for (t = 0; t < n; ++t) {
    double vals[2] = {A[t], B[t]};
    libxs_predict_push(NULL, model, vals, NULL);
}
libxs_predict_build(model, 0, 2, 0);
```

Non-stationary example (trending stock prices):

```
model = libxs_predict_create(WINDOW * 2, HORIZON);
libxs_predict_set_mode(model, LIBXS_PREDICT_TEMPORAL);
libxs_predict_set_series(model, 2, WINDOW);
libxs_predict_set_target(model, 0);
libxs_predict_set_decompose(model, LIBXS_PREDICT_SPREAD);
libxs_predict_set_diff(model, 0);
for (t = 0; t < n; ++t) {
    double vals[2] = {price_A[t], price_B[t]};
    libxs_predict_push(NULL, model, vals, NULL);
}
libxs_predict_build(model, 0, 2, 0);
```

### Training

```
int libxs_predict_push(libxs_lock_t* lock,
  libxs_predict_t* model,
  const double inputs[], const double outputs[]);
```

Push one training entry. When set_series was called and outputs is NULL, inputs holds nseries values representing one timestep; sliding windows are constructed at build time. Returns EXIT_SUCCESS or EXIT_FAILURE.

```
int libxs_predict_build(libxs_predict_t* model,
  int nclusters, int order, double quality);
```

Build the model from pushed entries. nclusters=0 selects sqrt(n) clusters automatically. order>0 uses at most that order, order=0 auto-optimizes, order<0 scans up to |order|.

The quality parameter (0..1) controls model compression and confidence scaling:

- quality=0: no compression, no coverage scaling (default).
- quality>0: leave-one-out pass removes entries that are perfectly predictable by their neighbors. Additionally, confidence is scaled by coverage at eval time: `conf = quality + coverage * (raw_conf - quality)` where coverage is the product of two factors:
  - inter-cluster: min(cluster_entries / expected_entries, 1). Penalizes underpopulated clusters.
  - intra-cluster: 1/(1 + query_dist/cluster_dmax). Penalizes queries far from the cluster center. The quality value acts as a confidence floor — predictions in sparse regions approach quality but never go below it. The quality is persisted in save/load.

```
int libxs_predict_build_task(libxs_lock_t* lock,
  libxs_predict_t* model, int nclusters, int order,
  double quality, int tid, int ntasks);
```

Per-thread collective form. All threads call with same parameters. tid=0 performs the build, others spin-wait.

### Evaluation

```
void libxs_predict_eval(libxs_lock_t* lock,
  const libxs_predict_t* model,
  const double inputs[], double outputs[],
  libxs_predict_info_t* info, int nblend);
```

Predict outputs for given inputs. nblend controls multi-cluster blending: 1=nearest only, 0=auto. The info pointer (optional) receives per-output confidence, variance, error bounds, and mode flags.

```
void libxs_predict_inverse(libxs_lock_t* lock,
  const libxs_predict_t* model,
  const double target_outputs[], double inputs[],
  libxs_predict_info_t* info);
```

Inverse prediction: find inputs that produce desired outputs.

```
void libxs_predict_eval_batch(
  const libxs_predict_t* model,
  const double inputs_batch[], double outputs_batch[],
  int count, int nblend);

void libxs_predict_eval_batch_task(
  const libxs_predict_t* model,
  const double inputs_batch[], double outputs_batch[],
  int count, int nblend, int tid, int ntasks);
```

Batch prediction. The task variant distributes queries across threads by slice.

### Query and Access

```
void libxs_predict_query(const libxs_predict_t* model,
  libxs_predict_query_t* info);

void libxs_predict_get(const libxs_predict_t* model,
  int index, double inputs[], double outputs[]);
```

query fills a statistics struct (cluster count, entry count, compression ratio, polynomial order, diff_order). get retrieves the i-th pushed entry (0-based).

`nscan` reports the entries in the largest cluster, which is what a neighbour query walks in the worst case: local evidence is gathered by scanning a cluster, so scoring cost grows with `nscan` rather than with `nentries`. Read it to know what a query costs before paying for it — a model whose partition is skewed, or built with a single cluster, puts most or all entries on that path.

### Probability

```
void* libxs_predict_prob_create(const libxs_predict_t* model);
void libxs_predict_prob_destroy(void* context);
int libxs_predict_prob_commit(libxs_predict_t* model,
  const void* context);

void libxs_predict_prob(libxs_lock_t* lock,
  const libxs_predict_t* model, void* context,
  const double inputs[], const double candidate[], double prob[],
  libxs_predict_prob_info_t* info, int vocabulary, int nblend);
int libxs_predict_prob_observe(libxs_lock_t* lock,
  const libxs_predict_t* model, void* context,
  const double inputs[], int output, const double* candidate,
  double values[], double probs[], int capacity, double* novel,
  libxs_predict_prob_info_t* info, int vocabulary, int nblend);
int libxs_predict_prob_support(const libxs_predict_t* model,
  int output, double values[], int capacity);
```

Where eval reports what the model would pick, these score a value the caller supplies — including one the model would never pick. `prob` is a point query for P(y|x); `prob_observe` reports the distribution over one discrete output and optionally observes the outcome, in that order, so weights cannot be advanced before the reported value is committed.

The escape weight mixing local evidence with the fallback prior is learned per output from realized log loss, and that learning is stream state rather than model state, so it lives in a context:

- `context != NULL` — adaptive; the weights adapt over the stream and the model is not modified.
- `context == NULL` — frozen; the model's stored weights are used and never written, so results do not depend on call order.

`prob_commit` copies a context's converged weights into the model. It is required before frozen scoring means anything: without it the model still holds its uniform prior, and freezing at that prior scores far worse than adapting.

`prob_support` returns the attested support of a discrete output (distinct values, sorted ascending), writing up to `capacity` entries and nothing at all when the support is larger. The support is fixed after build, so this reads it directly instead of through a scoring call.

### Persistence

```
int libxs_predict_save(const libxs_predict_t* model,
  void* buffer, size_t* size);
libxs_predict_t* libxs_predict_load(
  const void* buffer, size_t size);
```

Save: pass buffer=NULL to query required size, then allocate and call again. Load: returns a ready-to-eval model or NULL.

Load accepts every released format version (currently 1 and 2) and defaults fields an older file does not carry. A v1.0.0 flat model stores no global entry order, so it abstains from `libxs_predict_inverse` and from recency weighting, as it did under v1.0.0; saving it again writes version 2 without that order rather than inventing one. Version 2 adds a CRC32 trailer, so a truncated or corrupted file is rejected instead of silently loading a partial model.

### CSV Import

```
int libxs_predict_load_csv(libxs_predict_t* model,
  const char filename[], const char delims[],
  const char* inputs[], int ninputs,
  const char* outputs[], int noutputs);
```

Load delimited text and push entries. delims=NULL auto-detects separator. Returns number of entries pushed, or -1 on error.

### Structures

```
typedef struct libxs_predict_info_t {
  const double* values;
  const double* error;
  const double* confidence;
  const double* variance;
  const double* lower;
  const double* upper;
  const int* interpolated;
  int noutputs;
  int cluster;
  double distance;
} libxs_predict_info_t;
```

Populated by eval when info is non-NULL. confidence holds per-output confidence (0..1), incorporating:

- variety: kNN vote agreement (classify) or 1.0 (interpolation),
- coverage: cluster population and query centrality (when quality>0, see libxs_predict_build),
- consistency: round-trip penalty (when set_consistency>0). lower/upper hold per-output prediction intervals (NULL when set_quantile is 0, i.e. disabled). When enabled, bounds are the distance-weighted quantiles from k neighbors scaled by 1/confidence. variance holds per-output neighbor disagreement. cluster gives the assigned cluster index (-1 if blended). distance gives normalized distance to nearest centroid.

```
typedef struct libxs_predict_query_t {
  double compression;
  int order;
  int nclusters;
  int nentries;
  int iterations;
  int diff_order;
  int window;   /* effective sliding window (0 if series mode off) */
  int nbank;    /* window views built (1 when disabled) */
  int nscan;    /* entries in the LARGEST cluster */
} libxs_predict_query_t;
```

Populated by query. compression = raw/model size ratio. order = polynomial order used. diff_order = differencing order (0 if disabled).

## Registry

Header: `libxs_reg.h`

Thread-safe key-value store backed by a hash table with per-thread caching.

### Configuration Macros

| Macro | Default | Description |
|---|---|---|
| `LIBXS_REGKEY_MAXSIZE` | 64 | Maximum key size in bytes |
| `LIBXS_REGISTRY_NBUCKETS` | 64 | Initial hash-table buckets (power of two); registries grow dynamically |
| `LIBXS_REGCACHE_NENTRIES` | 16 | Thread-local cache entries per thread (power of two; 0 disables caching) |

### Types

```
typedef struct libxs_registry_t libxs_registry_t;   /* opaque */

typedef struct libxs_registry_info_t {
  size_t capacity, size, nbytes;
} libxs_registry_info_t;
```

### Lifetime

```
libxs_registry_t* libxs_registry_create(void);
```

Create a new registry. Returns `NULL` in case of an error.

```
void libxs_registry_destroy(libxs_registry_t* registry);
```

Destroy a registry and release all entries.

```
libxs_lock_t* libxs_registry_lock(libxs_registry_t* registry);
```

Return a pointer to the registry's internal lock. The returned lock can be passed as the `lock` argument to other registry functions (e.g., to hold the lock across multiple operations).

### Iteration

```
void* libxs_registry_begin(libxs_registry_t* registry,
  const void** key, size_t* cursor);
void* libxs_registry_next(libxs_registry_t* registry,
  const void** key, size_t* cursor);
```

Enumerate entries. Initialize `*cursor` to 0 before the first `begin` call. Each call returns the value pointer and writes the key pointer to `*key`, or returns NULL when iteration is complete.

### Access

```
void* libxs_registry_set(libxs_registry_t* registry,
  const void* key, size_t key_size,
  const void* value_init, size_t value_size,
  libxs_lock_t* lock /* = NULL */);
```

Insert or update a key-value pair. The key must be binary-reproducible (zero-initialize padding). `value_init` may be NULL to defer initialization. Re-registering an existing key reallocates if the new value is larger. When `lock` is NULL the registry's internal lock is used. Returns a pointer to the stored value.

```
void* libxs_registry_get(const libxs_registry_t* registry,
  const void* key, size_t key_size,
  libxs_lock_t* lock /* = NULL */);
```

Look up a value by key. Returns NULL if not found.

```
unsigned int libxs_registry_hash(const libxs_registry_t* registry,
  const void* key, size_t key_size);

void* libxs_registry_get_hashed(const libxs_registry_t* registry,
  const void* key, size_t key_size, unsigned int hash,
  libxs_lock_t* lock /* = NULL */);

void* libxs_registry_set_hashed(libxs_registry_t* registry,
  const void* key, size_t key_size, unsigned int hash,
  const void* value_init, size_t value_size,
  libxs_lock_t* lock /* = NULL */);
```

Hash a key once and reuse the value across several operations. The seed is per-registry (assigned in creation order, hence reproducible across runs and independent of the allocator), so a hash is only valid for the registry that produced it. The same value can also index caller-side tables keyed by the same shape. `libxs_registry_get/_set` are wrappers that hash internally.

`libxs_registry_get_hashed` additionally consults the thread-local lookup cache *before* taking the lock, so a repeated query of the same key costs no lock at all. That fast path is restricted to values too large for the registry's inline storage, whose heap buffer does not move when the table is rehashed, and it assumes the entry is neither removed nor resized while another thread queries it – true for a registry that only grows. `libxs_registry_get` is unchanged and still takes the lock whenever one is given.

```
int libxs_registry_get_copy(const libxs_registry_t* registry,
  const void* key, size_t key_size,
  void* value_out, size_t value_size,
  libxs_lock_t* lock /* = NULL */);
```

Thread-safe query: copies up to `value_size` bytes of the stored value into `value_out` under the lock. Unlike `libxs_registry_get`, the caller never receives a raw pointer into the registry's internal storage, making this safe under concurrent modifications. Returns non-zero if the key was found, zero otherwise.

```
int libxs_registry_has(libxs_registry_t* registry,
  const void* key, size_t key_size,
  libxs_lock_t* lock /* = NULL */);
```

Check whether a key exists. Non-zero if found.

```
size_t libxs_registry_value_size(libxs_registry_t* registry,
  const void* key, size_t key_size,
  libxs_lock_t* lock /* = NULL */);
```

Query the stored value size in bytes. Returns 0 if not found.

```
void libxs_registry_remove(libxs_registry_t* registry,
  const void* key, size_t key_size,
  libxs_lock_t* lock /* = NULL */);
```

Remove a key-value pair and release its memory.

```
int libxs_registry_extract(libxs_registry_t* registry,
  const void* key, size_t key_size,
  void* value_out, size_t value_size,
  libxs_lock_t* lock /* = NULL */);
```

Atomically retrieve and remove a key-value pair. Copies up to `value_size` bytes of the stored value into `value_out` (may be NULL to discard the value), then removes the entry and releases its memory. The lookup, copy, and removal are performed under a single lock hold, eliminating the race that exists when calling `libxs_registry_get` followed by `libxs_registry_remove` separately. Returns non-zero if the key was found, zero otherwise.

### Serialization

```
int libxs_registry_save(const libxs_registry_t* registry,
  void* buffer, size_t* size);
```

Save the registry to a binary buffer. Pass `buffer = NULL` to query the required size (written to `*size`). On success, `*size` contains the number of bytes written. Returns `EXIT_SUCCESS` or `EXIT_FAILURE`.

```
libxs_registry_t* libxs_registry_load(const void* buffer, size_t size,
  void (*fixup)(void* value, const void* key, size_t key_size,
    size_t value_size, void* udata),
  void* udata /* = NULL */);
```

Load a registry from a binary buffer (previously produced by `libxs_registry_save`). Keys are materialized immediately. When `fixup` is NULL, values remain in the buffer and are accessed on demand via `libxs_registry_get`; the buffer must remain valid for the lifetime of the returned registry. When `fixup` is non-NULL, each value is heap-allocated immediately and the callback is invoked once per entry, allowing the caller to restore pointers or handles by key identity. Returns a new registry or `NULL` on failure.

### Status

```
int libxs_registry_info(libxs_registry_t* registry,
  libxs_registry_info_t* info);
```

Query registry capacity, number of entries, and total bytes stored.

## Random Number Generator

Header: `libxs_rng.h`

Thread-safe pseudo-random number generation (SplitMix64 with per-thread TLS state).

### Matrix Initialization Macros

```
LIBXS_MATRNG(INT_TYPE, REAL_TYPE, ESPAN, DST, NROWS, NCOLS, LD, SCALE)
LIBXS_MATRNG_SEQ(INT_TYPE, REAL_TYPE, ESPAN, DST, NROWS, NCOLS, LD, SCALE)
LIBXS_MATRNG_OMP(INT_TYPE, REAL_TYPE, ESPAN, DST, NROWS, NCOLS, LD, SCALE)
```

Fill a column-major matrix (NROWS x NCOLS, leading dimension LD) with deterministic values. The `_OMP` variant parallelizes with OpenMP; `_SEQ` is serial.

ESPAN (exponent span) selects the initialization mode:

ESPAN == 0 Shuffle mode. A coprime permutation maps each linear index to a unique value in [-|SCALE|, +|SCALE|]. Covers the full LD*NCOLS range (including padding).

ESPAN != 0 Adversarial exponent-span mode for stress-testing floating-point emulation (Ozaki scheme, etc.). Base values are shuffled in [1, 2), then each column j is scaled by

```
  2^(sign(ESPAN) \* floor(|ESPAN| \* j / (NCOLS-1)))

so column 0 has exponent 0 and the last column has
exponent +/-|ESPAN|. Padding rows \[NROWS, LD) are
zero-filled.

Use +ESPAN for matrix A and -ESPAN for matrix B so
that A\*B remains well-conditioned while each operand
individually has adversarial exponent range.
```

Example — adversarial DGEMM test with exponent span 512:

```
int espan = 512;
LIBXS_MATRNG(int, double, +espan, a, m, k, lda, 1.0);
LIBXS_MATRNG(int, double, -espan, b, k, n, ldb, 1.0);
```

### Functions

```
void libxs_rng_set_seed(unsigned int seed);
```

Set the seed of the calling thread's PRNG state. Each thread maintains independent state via TLS. Unseeded threads start with a deterministic default (seed = 1).

```
unsigned int libxs_rng_u32(unsigned int n);
```

Return a pseudo-random value in [0, n) with uniform distribution (Lemire's nearly-divisionless method). Thread-safe.

```
double libxs_rng_f64(void);
```

Return a double-precision value in [0, 1) with full 53-bit mantissa resolution. Thread-safe.

```
void libxs_rng_seq(void* data, size_t nbytes);
```

Fill a buffer with pseudo-random bytes. Thread-safe.

## String Utilities

Header: `libxs_str.h`

Case-insensitive string search, edit distance, word-level matching and difference, similarity scoring, UTF-8 code point iteration, and value formatting.

### Substring Search

```
const char* libxs_stristrn(const char a[], const char b[],
  size_t maxlen);
const char* libxs_stristr(const char a[], const char b[]);
```

Case-insensitive substring search. `stristrn` limits the match length to `maxlen` characters of `b`. Returns a pointer to the first match in `a`, or NULL.

### Edit Distance

```
int libxs_stridist(const char a[], const char b[]);
```

Case-insensitive character-level edit distance (Levenshtein) between two strings. Returns the minimum number of single-character insertions, deletions, or substitutions to transform `a` into `b` (ignoring case). Returns -1 for NULL input.

### Word Matching

```
int libxs_strimatch(const char a[], const char b[],
  const char delims[], int* count);
```

Word-level fuzzy matching. Counts how many words in `a` (or `b`) appear in the other string (case-insensitive, symmetric). Optional `delims` define word separators (default: space, tab, semicolon, comma, colon, dash). Optional `count` receives the total word count of the larger string. Returns -1 for invalid input.

### Word Difference

```
int libxs_stridiff(const char a[], const char b[],
  const char delims[], int tolerance, int* count);
```

Word-level set difference with edit-distance tolerance. Counts how many words in the *smaller* string cannot be matched (within `tolerance` edits) to any word in the larger string. Matching is greedy (cheapest pair first), case-insensitive, and each word can be consumed at most once.

Parameters:

- `a`, `b` — input strings (NULL returns -1)
- `delims` — word separator characters (NULL uses default: space, tab, semicolon, comma, colon, dash)
- `tolerance` — maximum Levenshtein distance for two words to be considered a match (0 = exact match only, 1 = allows one edit such as plural/tense inflection)
- `count` — optional output, receives the word count of the larger string

Returns the number of unmatched words from the smaller string. A return value of 0 means every word in the shorter string found a match in the longer one. Returns -1 for NULL input.

Properties:

- Symmetric: `stridiff(a, b, ...)` equals `stridiff(b, a, ...)` because matching operates from the smaller side.
- Order-independent: word positions do not affect the result.
- Tolerance=0 gives exact word-level multiset difference.
- Tolerance=1 handles common morphological variation (plurals, verb tenses, e.g., "thread" matches "threads").

Example — measuring sentence redundancy:

```
int defect, total;
defect = libxs_stridiff(
  "The cache stores frequently accessed data.",
  "Frequently accessed data is kept in memory.",
  NULL, 1, &total);
/* defect=2 (unmatched: "kept", "memory"), total=7 */
/* redundancy = 1 - defect/total = 0.71 */
```

### Word Similarity

```
typedef enum libxs_strisimilar_t {
  LIBXS_STRISIMILAR_GREEDY,
  LIBXS_STRISIMILAR_TWOOPT,
  LIBXS_STRISIMILAR_DEFAULT = LIBXS_STRISIMILAR_GREEDY
} libxs_strisimilar_t;

int libxs_strisimilar(const char a[], const char b[],
  const char delims[], libxs_strisimilar_t kind, int* order);
```

Word-level similarity score combining edit distance and word-order analysis.

Strings are split into words using the same delimiters as `libxs_strimatch`. Each word in `a` is matched to a word in `b` via minimum-cost bipartite matching, where the cost of a pair is the character-level Levenshtein distance (case-insensitive). Unmatched words (when the strings have different word counts) contribute their full length.

The matching strategy is selected by `kind`:

- `GREEDY` — picks the globally cheapest pair first.
- `TWOOPT` — refines the greedy result by iteratively swapping pairs whenever a swap reduces total cost.

The optional `order` output receives the number of pairwise inversions among matched words (Kendall tau distance), measuring how much the word order differs (0 = same order).

Returns the total edit distance (0 for identical word sets in any order), or -1 for invalid input.

### Token Extraction

```
const char* libxs_strtoken(const char str[],
  const char delims[], int index, int* length);
```

Non-destructive access to the `index`-th token in a delimited string. Tokens are separated by characters in `delims` (default: comma). Leading and trailing whitespace within each token is trimmed. Returns a pointer into `str` at the token start, or NULL if `index` is out of range. Optional `length` receives the trimmed token length.

### UTF-8 Code Points

```
size_t libxs_utf8_size(const unsigned char text[], size_t size,
  size_t pos);
unsigned long libxs_utf8_decode(const unsigned char text[],
  size_t size, int* width);
```

Two entry points because the right answer for malformed input depends on the question being asked.

`libxs_utf8_size` is **lenient**: it reports the width the lead byte at `text[pos]` claims, clamped to the bytes that remain, and never less than 1 so a scan always advances. It does not inspect continuation bytes. Use it to walk text a code point at a time.

`libxs_utf8_decode` is **strict**: it returns the code point and writes the bytes consumed to `width` (may be NULL). A sequence that is truncated, or whose continuation bytes are not continuation bytes, yields the **lead byte** as the value and a width of 1. Use it when the code point value is wanted.

On well-formed text the two agree, and a scan by either covers the string exactly once. They diverge only on malformed input, where the lenient form skips the claimed span and the strict form skips one byte:

| Input | `utf8_size` | `utf8_decode` |
|---|---|---|
| `41` | 1 | U+0041, width 1 |
| `C3 A4` | 2 | U+00E4, width 2 |
| `E2 80 99` | 3 | U+2019, width 3 |
| `C3 78` | 2 | U+00C3, width 1 |
| `C3` (alone) | 1 (clamped) | U+00C3, width 1 |

The strict rule exists because a caller that tests a property of the value — is this a vowel, is this punctuation — must not be handed a code point assembled from bytes that do not belong to it.

### Value Formatting

```
size_t libxs_format_value(char buffer[], int buffer_size,
  size_t nbytes, const char scale[], const char* unit, int base);
```

Format a scalar value with SI-style scaling. Example:

```
libxs_format_value(buf, sizeof(buf), nbytes, "KMGT", "B", 10);
```

produces a human-readable byte count such as “128 KB”. Returns the value in the selected unit so the caller can decide whether to print the buffer.

### Relationship Between Functions

The string utilities form a hierarchy of comparison granularity:

| Function | Granularity | Returns | Use case |
|---|---|---|---|
| `stridist` | characters | edit distance | spelling similarity |
| `strimatch` | words | matched word count | overlap detection |
| `stridiff` | words | unmatched count | redundancy measurement |

| Function | Granularity | Returns | Use case |
|---|---|---|---|
| `strisimilar` | words | total edit cost | structural similarity |

`stridiff` is the word-level analog of the byte-level `libxs_setdiff` (from `libxs_math.h`). Where `setdiff` counts unmatched elements in numeric arrays within a tolerance, `stridiff` counts unmatched words within an edit-distance tolerance. Both are order-independent and symmetric.

## Synchronization

Header: `libxs_sync.h`

Thread-local storage, atomic operations, lock abstractions (spin/mutex/rwlock/atomic), file locking, and stdio synchronization.

### Thread-Local Storage

`LIBXS_TLS`

Storage-class qualifier for thread-local variables. Maps to `__thread`, `__declspec(thread)`, or `thread_local` depending on the compiler. Defined as empty when TLS is unavailable or disabled (`LIBXS_NO_TLS`).

### Atomic Operations

The header provides a suite of macros for atomic loads, stores, compare-and-swap, and arithmetic. The implementation selects among GCC builtins (`__atomic_*`), legacy GCC sync builtins, Windows Interlocked intrinsics, or no-op fallbacks depending on the compiler and `LIBXS_SYNC` setting.

| Macro | Description |
|---|---|
| `LIBXS_ATOMIC_LOAD(SRC, KIND)` | Atomic load |
| `LIBXS_ATOMIC_STORE(DST, VALUE, KIND)` | Atomic store |
| `LIBXS_ATOMIC_STORE_ZERO(DST, KIND)` | Atomic store of zero |
| `LIBXS_ATOMIC_CMPSWP(DST, OLDVAL, NEWVAL, KIND)` | Compare-and-swap |
| `LIBXS_ATOMIC_FETCH_OR(DST, VALUE, KIND)` | Fetch-and-or |
| `LIBXS_ATOMIC_FETCH_ADD(DST, VALUE, KIND)` | Fetch-and-add |
| `LIBXS_ATOMIC_FETCH_SUB(DST, VALUE, KIND)` | Fetch-and-subtract |
| `LIBXS_ATOMIC_ADD_FETCH(DST, VALUE, KIND)` | Add-and-fetch |
| `LIBXS_ATOMIC_SUB_FETCH(DST, VALUE, KIND)` | Subtract-and-fetch |
| `LIBXS_ATOMIC_TRYLOCK(DST, KIND)` | Try-lock (returns acquired state) |
| `LIBXS_ATOMIC_ACQUIRE(DST, NPAUSE, KIND)` | Spin-acquire with backoff |
| `LIBXS_ATOMIC_RELEASE(DST, KIND)` | Release (store zero) |
| `LIBXS_ATOMIC_SYNC(KIND)` | Full memory fence |

The `KIND` parameter selects the memory order but is ignored on most backends (sequential consistency is always used).

### Lock Abstraction

```
LIBXS_LOCK_TYPE(KIND)
LIBXS_LOCK_INIT(KIND, LOCK, ATTR)
LIBXS_LOCK_DESTROY(KIND, LOCK)
LIBXS_LOCK_ACQUIRE(KIND, LOCK)
LIBXS_LOCK_TRYLOCK(KIND, LOCK)
LIBXS_LOCK_RELEASE(KIND, LOCK)
```

Generic lock interface parameterized by `KIND`:

| KIND | Backend |
|---|---|
| `LIBXS_LOCK_SPINLOCK` | Atomic spin-lock |
| `LIBXS_LOCK_MUTEX` | Pthreads / Windows CRITICAL_SECTION |
| `LIBXS_LOCK_RWLOCK` | Pthreads reader-writer lock |
| `LIBXS_LOCK_ATOMIC` | Lightweight atomic lock |

The default lock kind used by the library is `LIBXS_LOCK` (resolves to one of the above based on build configuration).

```
typedef LIBXS_LOCK_TYPE(LIBXS_LOCK) libxs_lock_t;
```

General-purpose lock type. Instances of `libxs_lock_t` are used by the registry and other library components; users may also create their own.

Reader-writer variants are available for `LIBXS_LOCK_RWLOCK`:

```
LIBXS_LOCK_ACQREAD(KIND, LOCK)
LIBXS_LOCK_RELREAD(KIND, LOCK)
LIBXS_LOCK_TRYREAD(KIND, LOCK)
```

### File Locking

```
LIBXS_FLOCK(FILE)
LIBXS_FUNLOCK(FILE)
```

Per-file locking for thread-safe I/O. Maps to `flockfile`/`funlockfile` on POSIX, `_lock_file`/`_unlock_file` on Windows, or no-ops when synchronization is disabled.

### Functions

```
unsigned int libxs_nranks(void);
```

Return the number of MPI ranks.

```
unsigned int libxs_nrank(void);
```

Return the MPI rank of the calling process.

```
unsigned int libxs_rid(void);
```

Return a rank ID of the calling process.

```
unsigned int libxs_pid(void);
```

Return the process ID of the calling process.

```
unsigned int libxs_tid(void);
```

Return a zero-based, consecutive thread ID for the calling thread. TID = 0 does not necessarily correspond to the main thread.

```
void libxs_stdio_acquire(void);
void libxs_stdio_release(void);
```

Acquire/release a global lock around console output. The macros `LIBXS_STDIO_ACQUIRE()` and `LIBXS_STDIO_RELEASE()` expand to these calls.

## Timer

Header: `libxs_timer.h`

High-resolution timing via a calibrated TSC when available, with an OS real-time clock fallback.

### Types

```
typedef unsigned long long libxs_timer_tick_t;
```

Integer type representing a single timer sample.

```
typedef struct libxs_timer_info_t {
  int tsc;   /* non-zero if a calibrated TSC is used */
} libxs_timer_info_t;
```

### Functions

```
int libxs_timer_info(libxs_timer_info_t* info);
```

Query timer properties. `info->tsc` is non-zero when RDTSC (x86), CNTVCT_EL0 (AArch64), or MFTB (PPC64) is available. Triggers lazy library initialization if necessary.

```
libxs_timer_tick_t libxs_timer_tick(void);
```

Sample the current tick of the monotonic timer. The tick unit is platform-specific; convert with `libxs_timer_duration` or `libxs_timer_ncycles`.

```
libxs_timer_tick_t libxs_timer_ncycles(
  libxs_timer_tick_t tick0, libxs_timer_tick_t tick1);   /* inline */
```

Unsigned difference `tick1 - tick0`, handling wrap-around safely. Implemented via `LIBXS_DELTA`.

```
double libxs_timer_duration(
  libxs_timer_tick_t tick0, libxs_timer_tick_t tick1);
```

Convert a pair of ticks to elapsed wall-clock time in seconds.

## Tokenizer

Header: `libxs_token.h`

The tokenizer provides reversible metatokens and vocabulary-backed lexemes. These representations serve separate APIs:

- reversible metatokens for prediction and exact text reconstruction;
- vocabulary-backed lexemes for interpretation, matching, and rules.

### Metatoken Layout

`libxs_token_t` is one fixed 8-byte physical cell:

```
bit 7      sentence boundary on the final cell
bit 6..4   generic payload kind
bit 3      another cell continues this logical metatoken
bit 2..0   payload bytes in this cell, 1..7
bytes 1..7 payload
```

A logical metatoken may occupy several cells. Long text units therefore do not require a larger struct or an external vocabulary. `libxs_token_span` validates the continuation chain, while `libxs_token_read` reconstructs its payload and metadata.

Encoder-produced cells are zero-initialized, including unused payload bytes. Binary comparison therefore remains deterministic when control metadata matters. For content matching, use `libxs_token_payload_equal` on physical cells or `libxs_token_payload_match` on complete logical metatokens. These functions compare declared payload length and bytes while ignoring kind, continuation, sentence flags, unused trailing bytes, and (for logical matching) cell layout. Do not infer payload length from trailing zeros; zero is a valid payload byte.

Payload kinds are independent of segmentation policy: word and syllable settings both emit `LIBXS_TOKEN_TEXT`. Their boundaries differ, but consumers use the same read and decode APIs.

### Tokenizer Configuration

Granularity belongs to the tokenizer, not to each encode call:

```
libxs_tokenizer_t* tokenizer =
  libxs_tokenizer_create(LIBXS_TOKEN_GRANULARITY_WORD);
libxs_token_stream_t stream;

libxs_token_stream_init(&stream);
libxs_token_stream_encode(tokenizer, &stream, text, size);
libxs_tokenizer_set_granularity(tokenizer,
  LIBXS_TOKEN_GRANULARITY_SYLLABLE);
```

Available policies are native chunks, words, and syllables. Encoding is otherwise policy-neutral, and every policy preserves all input bytes.

The stream is self-describing. Decoding does not require the tokenizer or a lexicon:

```
libxs_token_stream_decode(&stream, &decoded, &decoded_size);
```

For every successful encoding:

```
decode(encode(text)) == text
```

including whitespace, spelling, case, punctuation, numbers, and UTF-8 bytes.

### Logical Iteration

Physical cells are traversed by logical span:

```
size_t pos = 0;
while (pos < stream.size) {
  libxs_token_info_t info;
  unsigned char payload[64];
  if (EXIT_SUCCESS != libxs_token_read(stream.data, stream.size, pos,
    payload, sizeof(payload), &info)) break;
  pos += info.cells;
}
```

`info.kind` describes text, number, whitespace, punctuation, markup, literal, reference, or control payload. `info.is_sentence` is carried by the final cell of a logical metatoken.

### Lexical API

`libxs_lexeme_t` remains an independent 8-byte lexical occurrence:

```
typedef struct libxs_lexeme_t {
  unsigned int id;
  unsigned short length;
  unsigned short flags;
} libxs_lexeme_t;
```

`libxs_lexeme_stream_encode` normalizes words, maps them through a persistent lexicon, applies caller-owned normalization and classification rules, and emits stable IDs. This representation is intentionally interpretive rather than reversible. It is suitable for grounded QA, lexical matching, and rule evaluation, but raw-text likelihood should use metatokens.

## Low-Level Utilities and SIMD

Header: `libxs_utils.h`

Target-attribute machinery, ISA feature gates, intrinsic fix-ups, bit-scan operations, and AVX-512 inline helpers.

### Target Attributes

```
LIBXS_INTRINSICS(TARGET)
```

Expands to a `__attribute__((target(...)))` clause for the given ISA level. Used to compile individual functions for a higher ISA than the baseline without changing compiler flags. The macro is a no-op when the baseline already covers the requested level or when the compiler does not support multi-versioning.

```
LIBXS_ATTRIBUTE_TARGET(TARGET)
```

Lower-level macro that resolves `TARGET` (an integer ISA constant such as `LIBXS_X86_AVX512`) to the corresponding target string, e.g. `target("avx2,fma,avx512f,...")`.

### ISA Feature Gates

Preprocessor symbols defined when the compiler can generate code for a given ISA:

| Macro | ISA level |
|---|---|
| `LIBXS_INTRINSICS_X86` | x86 baseline (SSE2) |
| `LIBXS_INTRINSICS_SSE3` | SSE3 |
| `LIBXS_INTRINSICS_SSE42` | SSE4.2 |
| `LIBXS_INTRINSICS_AVX` | AVX |
| `LIBXS_INTRINSICS_AVX2` | AVX2 + FMA |
| `LIBXS_INTRINSICS_AVX512` | AVX-512 (F + CD + DQ + BW + VL + VNNI) |

LIBXS_STATIC_TARGET_ARCH records the highest ISA provided by the compiler flags. LIBXS_MAX_STATIC_TARGET_ARCH records the highest ISA reachable via target attributes. Target attributes (LIBXS_ATTRIBUTE_TARGET_*) are defined up to LIBXS_X86_AVX10_512 depending on compiler version.

### Intrinsic Fix-ups

Portability wrappers for intrinsics whose signatures differ across compilers:

| Macro | Purpose |
|---|---|
| LIBXS_INTRINSICS_LOADU_SI128 | _mm_loadu_si128 |
| LIBXS_INTRINSICS_MM512_LOAD_PS/PD | Unaligned 512-bit float/double load |
| LIBXS_INTRINSICS_MM512_STREAM_* | Non-temporal 512-bit stores |
| LIBXS_INTRINSICS_MM256_STORE_EPI32 | 256-bit integer store |
| LIBXS_INTRINSICS_MM512_SET_EPI16 | Portable 512-bit set from 32 int16 values |
| LIBXS_INTRINSICS_MM512_UNDEFINED_* | Undefined-value initializers (zero in debug) |
| LIBXS_INTRINSICS_MM512_EXTRACTI64X4_EPI64 | 256-bit extract from 512-bit |
| LIBXS_INTRINSICS_MM512_ABS_PS | Absolute value of packed floats |
| LIBXS_INTRINSICS_MM512_MASK_I32GATHER_EPI32 | Masked gather |
| LIBXS_INTRINSICS_MM512_STORE/LOAD_MASK* | Mask register I/O (8/16/32/64-bit) |
| LIBXS_INTRINSICS_MM512_CVTU32_MASK* | Convert uint32 to mask register |

### Bit-Scan Operations

```
LIBXS_INTRINSICS_BITSCANFWD32(N)   /* count trailing zeros, 32-bit */
LIBXS_INTRINSICS_BITSCANFWD64(N)   /* count trailing zeros, 64-bit */
LIBXS_INTRINSICS_BITSCANBWD32(N)   /* bit index of highest set bit, 32-bit */
LIBXS_INTRINSICS_BITSCANBWD64(N)   /* bit index of highest set bit, 64-bit */
```

Use GCC __builtin_ctz/__builtin_clz when available, Windows _BitScanForward/_BitScanReverse intrinsics on MSVC, or portable software fallbacks (*_SW variants). All return 0 when N is 0.

### Derived Macros

```
LIBXS_NBITS(N)     /* minimum bits to represent N */
LIBXS_ISQRT2(N)    /* fast power-of-two approximation of sqrt(N) */

unsigned int LIBXS_ILOG2(unsigned long long n);   /* ceil(log2(n)) */
```

LIBXS_ILOG2 is a function (not a macro) and handles the n = 0 case.

### MXCSR (FP Environment)

Macros for preserving and restoring the x86 MXCSR register (floating-point control/status). On non-x86 platforms these are no-ops.

| Macro | Description |
|---|---|
| `LIBXS_MXCSR_FTZ` | Flush-to-zero flag (0x8000 on x86, 0 otherwise) |
| `LIBXS_MXCSR_DAZ` | Denormals-are-zero flag (0x0040 on x86, 0 otherwise) |
| `LIBXS_MXCSR_GET()` | Read the MXCSR register (`_mm_getcsr()` on x86, 0 otherwise) |
| `LIBXS_MXCSR_SET(VALUE)` | Write the MXCSR register (`_mm_setcsr()` on x86, no-op otherwise) |

### AVX-512 Inline Functions

Available only when `LIBXS_INTRINSICS_AVX512` is defined.

```
__m512i libxs_mulhi_epu32(__m512i a, __m512i b);   /* inline */
```

Unsigned 32-bit high-multiply for 16 lanes: floor(a * b / 2^32). Emulates the missing `_mm512_mulhi_epu32` via even/odd `_mm512_mul_epu32`.

```
__m512i libxs_mod_u32x16(__m512i x,
  unsigned int p, unsigned int rcp);               /* inline */
```

Vectorized Barrett reduction: x mod p for 16 uint32 lanes. `rcp` is the Barrett reciprocal from `libxs_barrett_rcp()`. This is the SIMD counterpart of the scalar `libxs_mod_u32`.

## Appendix

### Presentations

- Self-Diagnosing Parameter Prediction
- Practical Ozaki Schemes

### Scripts

#### Build Support

**tool_source.sh** — Generate `libxs_source.h` (header-only amalgamation). Called by `make headers`.

**tool_version.sh** — Extract version from Git tags. Generates `libxs_version.h`.

**libxs.pc.in** — Template for the installed `.pc` file.

### Code Quality

**tool_analyze.sh** — Run cppcheck on `src/`.

**tool_checkabi.sh** — Compare exported symbols against a reference.

**tool_clangformat.sh** — Run clang-format and shellcheck.

**tool_normalize.sh** — Strip trailing whitespace, fix line endings.

### Documentation

**md2pptx.py** — Convert Markdown to PowerPoint. Splits on `---` rules or `##` headings (auto-detected). Requires `python-pptx` and `lxml`.

### Development

**tool_cpuinfo.sh** — Print CPU topology (sockets, cores, threads).

**tool_getenvars.sh** — List environment variables used in the source tree.

**tool_pexec.sh** — Parallel command execution with CPU affinity (`-h` for options).

**tool_gitaddforks.sh** — Add GitHub forks as Git remotes.

**tool_gitprune.sh** — Aggressive Git housekeeping (gc, fsck, repack).

## Contributing

This is the contribution policy for LIBXS and LIBXSTREAM. Both projects follow the same policy, so the text is project-neutral: it is maintained in LIBXS (`documentation/libxs_contrib.md`) and copied into dependent projects by `make policies`, the same way `Makefile.inc` is shared. Please edit it in LIBXS — the copy is overwritten whenever the original changes.

The code base is small, long-lived, and read far more often than it is written, so uniformity is what keeps diffs reviewable. When a rule below does not answer your case, the strongest rule is: **match the surrounding code**.

Not every rule is derived from first principles. Some are taste, some are habit, and some are plain superstition. They are the policy regardless: a uniform code base is worth more than an individually optimal choice, and rules that are merely arbitrary are still cheap to follow.

### Character Encoding and Whitespace

| Files | Encoding |
|---|---|
| Source, headers, kernels, Makefiles, scripts, YAML, plain text | US-ASCII only |
| Markdown (`*.md`) | any UTF-8 |

No Unicode in code or build files — no typographic quotes, no em dashes, no non-breaking spaces. Such characters survive editors, diffs, and generators poorly, and neither the compiler nor `sed` is required to handle them.

Markdown has no such restriction: a spaced em dash (—) instead of --, an en dash for numeric ranges (1–4), and mathematical notation (alpha, <=, ceil(n/2), x, sum) are all welcome wherever they read better than an ASCII transliteration.

Beyond encoding:

- **No tabs**, except where a Makefile requires them.
- LF line endings. CRLF is rejected.
- No trailing whitespace.
- No whitespace before `#` in a preprocessor directive.
- **No French spacing** in either sense: no space before `,`, `;`, `:`, `!`, or `?`, and a single space after a sentence-ending period. This holds for code, comments, commit messages, and documentation alike.
- A script carrying a shebang is executable in the index (mode 755); every other file is 644.

`scripts/tool_normalize.sh` checks, and where it can fixes, everything above except the spacing conventions. Run it before submitting — continuous integration does not.

### C Source File Structure

A translation unit is strictly grouped, in this order:

1. Includes
2. Macros
3. Types (typedefs, struct definitions)
4. Translation-unit variables (file-scope statics and globals)
5. Functions

No interleaving. A macro used only by the last function still belongs in the macro section. This makes the shape of every file predictable: what it depends on, what it configures, what state it holds, and what it does — in that order.

Every file carries the SPDX license header (BSD-3-Clause) verbatim as found in existing files.

## C Dialect

**C89 (ANSI C) in general.** The pre-submit configuration (`PEDANTIC=2`) compiles with `-std=c89`, so a C99-only construct breaks the build for everyone else even when it compiles for you:

- Declarations at the beginning of a block, before any statement. No declarations mixed into the middle of a block, and no declaration in a `for` initializer.
- `/* ... */` comments only.
- No variable-length arrays, compound literals, designated initializers, `long long`, `restrict`, or `//`-style line continuation tricks.
- Newer facilities are reached through the macros and typedefs the public headers already provide, not by raising the dialect locally.

Where a C99 (or later) construct is genuinely required, it is guarded and confined, in the same style as the existing guards.

## Functions

- **A function has a single exit.** No early `return`, no multiple return paths, no `goto`.
- Use a `result` variable, gate subsequent work on `EXIT_SUCCESS == result`, and return `result` at the single exit point.
- Constants go on the left-hand side of a comparison (`EXIT_SUCCESS == result`, `NULL != ptr`), which turns an accidental assignment into a compile error.
- Two blank lines between function definitions.
- At most one blank line inside a function body.

```
int example(const void* input, void** output) {
  int result = EXIT_SUCCESS;
  if (NULL == input || NULL == output) result = EXIT_FAILURE;
  if (EXIT_SUCCESS == result) {
    result = prepare(input);
  }
  if (EXIT_SUCCESS == result) {
    result = finish(input, output);
  }
  return result;
}
```

## Comments

- **A comment adds value on top of the code.** Never document what the code obviously does — the code already says that. Document what it cannot say: why this way, what breaks otherwise, which assumption is being relied on.
- Prefer no comment at all. Then prefer one line.
- **Single-line comments by default.** A multi-line comment has to earn its size: it is reserved for what is absolutely necessary, i.e., a risk or a trap worth spelling out — typically something that has already been gotten wrong once and would be gotten wrong again without the warning. Everything else is one line or nothing.
- API documentation in public headers is the other place a multi-line block belongs.
- `/* ... */` only. **C++ comments (`//`) are rejected** in `.c` and `.h`.
- **No decorative or banner-style comment blocks.** No `/*==== Section ====*/`, no boxes, no ASCII rules. The license header is the sole exception.
- Never write -- inside a comment (the ASCII rule keeps the em dash out, and a double hyphen reads as a typo); rephrase instead.

### Blank Lines

- Never three or more consecutive blank lines, anywhere.
- Exactly two blank lines separate function definitions.
- At most one blank line separates logical blocks inside a function.

### Formatting

A `.clang-format` file is present (LLVM-based, 2-space indent, 96-column limit, never tabs), and `scripts/tool_clangformat.sh` selects the newest available version of the tool.

**Do not bulk-reformat as part of a change.** Recent clang-format versions reflow entire files, which buries a small change in hundreds of unrelated lines and makes review impossible. This is why `tool_normalize.sh` has its reformat step deliberately disabled. Format new and edited code by hand to match the file around it; the formatter is a maintenance tool, run deliberately and committed on its own.

Do not mix reformatting, renaming, and behavioural change in one commit.

### Library Code

- Library code does not terminate the process: no `exit(...)` in `src/`. Return a status and let the caller decide.
- Environment variables carry the project's own prefix (`LIBXS_*` or `LIBXSTREAM_*`). `scripts/tool_getenvars.sh` lists what the source reads.
- **Header-only mode must keep working.** The amalgamated header (`*_source.h`, or the corresponding `-D*_SOURCE`) has to be includable from multiple translation units, so a new file-scope symbol in `src/*.c` needs internal linkage or the established macro treatment.
- Fix the cause, not the symptom. When a sample, test, or dependent project runs into a limitation of the library, change the library rather than working around it at the call site.

### Fortran

Fortran sources are **fixed-form** (`.f`). Free-form (`.f90`) is not used; please do not propose converting them.

### Scripts

- POSIX-portable shell. `sed -i` is rejected: it is not portable (macOS).
- Shell scripts pass `shellcheck`.
- Python is formatted with `black -l79` and passes `flake8` and `mypy`.

### Documentation

Documentation is terse and written for the person *using* the code, not for the person who wrote it.

- **Every `README.md` is user documentation**: what the thing does, how to enable it, how to run it, which environment variables and build knobs exist. Nothing more.
- Insights stay out. Design rationale, derivations, and performance analysis do not belong in a README — the single exception being a surprising usability implication the user has to know about (a knob that silently changes accuracy, a mode that only works on one vendor). If it does not change how someone uses the code, it is not user documentation.
- A short *why* belongs in the commit message.
- Document every new environment variable. A variable that `tool_getenvars.sh` reports but the documentation does not mention is a defect.
- A new page under `documentation/` needs a `nav` entry in `mkdocs.yml`.

Parts of `documentation/` are **generated**, and editing the output is lost work: the landing page comes from `README.md`, the development page from `scripts/README.md`, one page per sample from `samples/*/README.md`, one page per test from `tests/*.c`, this page from LIBXS, and the PDFs from all of the above. Version and amalgamated headers are generated too. Edit the source, then:

```
make documentation   # PDFs
make mkdocs          # serve the site with live reload
make mkslides        # serve a slide deck (SLIDES=<topic>)
```

The Makefile is authoritative about what is generated from what.

### Build Systems

**GNU Make is primary.** The `Makefile` and the shared `Makefile.inc` define the defaults, the knobs, and the behaviour; they are the reference. **CMake is secondary**: it has to produce the same artifacts, but it does not get to define anything.

- A new source file, build knob, or changed default lands in the Makefile first, then in `CMakeLists.txt`. `CMakeLists.txt` mirrors the Makefile's defaults and says so where it matters — when a default moves, move both.
- Every step is taken to keep the two interchangeable *from the consumer's side*. In particular, `make` writes the files a CMake consumer expects — the package configuration under `lib/cmake/<project>/` alongside the pkg-config `.pc` files — so a tree built and installed with GNU Make is usable via `find_package()` without CMake ever having run. GNU Make pretending to be CMake is a feature, not a workaround.
- Continuous integration builds and installs both ways and then consumes the result via `find_package()` and pkg-config. A change that only works in one of the two is incomplete.

### Building and Testing

Build without `DBG=1` by default. Debug builds are `-O0`, which makes any runtime-bearing sample prohibitively slow — never measure performance with one.

```
make -j $(nproc)                    # default build
make -j $(nproc) test               # test suite
make -j $(nproc) DBG=1 PEDANTIC=2   # correctness check before submitting
```

`PEDANTIC=2` enables strict warnings, `ANALYZE=1` runs the compiler's static analyzer, and `scripts/tool_analyze.sh` runs cppcheck. A change is expected to be warning-free under `DBG=1 PEDANTIC=2`, because that is what continuous integration builds: GCC, Intel oneAPI, and macOS, covering release and strict debug configurations as well as a header-only build compiled as C++. See `.github/workflows/` for the exact matrix.

## ABI and Versioning

The version derives from Git tags and `version.txt`; the shared library carries an SOVERSION. `scripts/tool_checkabi.sh` compares exported symbols against the recorded baseline. Do not remove or rename a published symbol — add a new one instead. Run the checker on a build that has symbol information:

```
make STATIC=0 SYM=1
scripts/tool_checkabi.sh
```

A symbol name outside the project's namespace is an error, not a warning.

## Commits and Pull Requests

- One concern per commit. Keep unrelated formatting out of it.
- Subject line: short, capitalized, no trailing period, e.g. `Improved device memory allocation`. An area prefix is fine: `CMake: updated to test kernels`.
- Explain *why* in the body when the subject cannot carry it.
- Contributions are accepted under the project's BSD-3-Clause license.

Before submitting:

```
scripts/tool_normalize.sh                 # whitespace, encoding, lint
make -j $(nproc) DBG=1 PEDANTIC=2 test    # strict build and tests
scripts/tool_checkabi.sh                  # only if public symbols changed
```

## Workflow with Assistants

If an AI assistant is used, the same policies apply, plus two more:

- Discuss design options and trade-offs before implementing.
- Present options as inline text, not as interactive choice widgets.

`CLAUDE.md` at the repository root is the machine-readable form of the coding conventions on this page, and is shared between the projects the same way. The two must agree; when a policy changes, change both — in LIBXS.